

XAI-SDN: An explainable entropy-guided machine learning framework for real-time DDoS detection in software defined networks

ADEEL AHMAD¹, ARSHAD ALI¹, RAFI US SHAN², and GAHANGIR HOSSAIN³

¹Faculty of Computer and Information Systems, Islamic University of Madinah, Al Madinah Al Munawarah, Saudi Arabia

²Faculty of Computer Information Science, Higher Colleges Of Technology, Al Mizn - Baniyas North - Abu Dhabi, United Arab Emirates

³Anuradha and Vikas Sinha Department of Data Science, University of North Texas, Denton, TX, USA, 76203-5017

Corresponding authors: Rafi us Shan (e-mail: rshan@hct.ac.ae) and Adeel Ahmad (e-mail: 443057803@stu.iu.edu.sa).

ABSTRACT Software-defined networking concentrates forwarding decisions in a logically centralized controller, which simplifies management and creates a single point of failure. A distributed denial-of-service campaign aimed at the controller, or one that merely generates enough table-miss events to saturate the control channel, can disable forwarding across an entire domain. Machine learning detects such traffic well, but an operator who cannot see why a flow was flagged will not authorize the system to act on its own. We present XAI-SDN, a controller-embedded detector that augments CICFlowMeter flow statistics with eight windowed Shannon entropy features maintained at expected amortized $\mathcal{O}(1)$ cost per update, classifies with a Random Forest, and attributes alerts with exact TreeSHAP under an explicit explanation budget.

Evaluated on the SYN partition of CIC-DDoS2019 under a chronological split, in which every training flow precedes every test flow, XAI-SDN attains 99.76% accuracy and 97.88% macro F1 over 1,077,238 held-out flows, with a false positive rate of 0.095% and an average precision of 0.99999. Because a random split of the same data instead yields 99.95% macro F1, we report both protocols and quantify the difference rather than selecting the more favorable one.

Two operational contributions follow. Attributing every positive prediction is untenable when almost all traffic is hostile: it reduces sustained throughput from 35,715 to 661 flows/s. We define the explanation trigger formally as an admission decision and show that computing one exact attribution per attack episode, with every other alert in that episode carrying a reference to its representative, restores 35,652 flows/s while leaving every alert resolvable to an attribution. The inherited attribution is exact for the representative flow and an episode-level summary for the rest, and we state throughout which of the two an alert carries. We also run the detector inside a live OpenFlow 1.3 control plane against Mininet, reporting controller CPU and memory, end-to-end detection latency, the offered rate at which a single controller saturates, and the flow-rule aggregation that keeps a bounded TCAM from overflowing during a flood. Supporting analyses cover a leakage audit with a label-permutation control, collinearity and attribution-stability diagnostics, the numerical behavior of the streaming entropy engine over 10^7 updates, and transfer to other attack vectors, a second capture day, InSDN and CIC-IDS2017. We also report where the entropy augmentation does and does not pay. In distribution it does not: the eighty exported flow statistics alone reach a slightly higher macro F1 than the full 88-dimensional vector. Under distribution shift it does, wherever the full feature set is computable, raising macro F1 from 0.6426 to 0.8364 on a held-out attack vector, from 0.4212 to 0.4820 across capture days, and from 0.2383 to 0.3811 on InSDN. On the one corpus whose export allows only four of the eight entropy features, a partial summary is worse than none.

INDEX TERMS DDoS detection, software defined networks, explainable AI, Shannon entropy, random forest, SHAP, network security, evaluation methodology

I. INTRODUCTION

SOFTWARE-defined networking (SDN) separates the data plane from the control plane and concentrates for-

warding decisions in a logically centralized controller [1], [2]. Enterprise and cloud operators adopt the architecture because a single control point simplifies reconfiguration, ad-

mits network-wide visibility, and makes traffic engineering programmable. The same concentration creates an obvious target. A distributed denial-of-service (DDoS) campaign directed at the controller, or one that simply generates enough table-miss events to saturate the control channel, can disable forwarding across the entire domain.

DDoS traffic takes several forms: UDP amplification, TCP SYN flooding, ICMP sweeps, and application-layer attacks such as HTTP floods and Slowloris [3], [4], [5]. Machine learning detects these patterns well, but most deployed detectors are opaque. An operator who cannot see why a flow was flagged cannot adjudicate a borderline alert, cannot defend an automated mitigation to an incident review, and in practice will not authorize the system to act autonomously [6], [7], [8], [9].

Three requirements therefore apply at once: accurate detection, processing cheap enough to keep pace with the control channel, and decisions an operator can inspect. Systems in the literature typically satisfy two [10], [11]. The difficulty is not that any one requirement is hard in isolation but that the third is usually paid for by sacrificing one of the others, and the size of that payment is rarely measured.

This paper measures it. We present XAI-SDN, a controller-embedded detector that augments CICFlowMeter flow statistics with windowed Shannon entropy features maintained at expected amortized constant cost per update, classifies with a Random Forest, and attributes alerts with exact TreeSHAP under an explanation budget: every admitted alert receives an attribution computed for that specific flow, and every alert the budget declines carries a reference to the exact attribution of its episode representative. Section III-B fixes the distinction and Section VII-K measures what it buys. The contribution is less the combination itself than the operational accounting that surrounds it, and the methodological care that the accounting turned out to require.

Our contributions are as follows.

- **A numerically characterized streaming entropy engine.** We give the update recurrence the implementation actually executes, state its complexity as expected amortized $\mathcal{O}(1)$ per admit/evict pair under explicit data-structure assumptions rather than as an unqualified bound, and measure the floating-point drift it accumulates over 10^7 updates together with the number of classifier decisions that drift changes.
- **A formally defined and budgeted explanation trigger.** Attributing every positive prediction is untenable when almost all traffic is hostile. We define the trigger once, measure the throughput collapse it causes, and quantify the throughput recovered by episode-level deduplication, a token-bucket budget, and off-path asynchronous attribution, reporting the resulting throughput versus explanation-coverage frontier.
- **A control-plane cost accounting.** Using an OpenFlow 1.3 testbed in which the detector runs inside a live controller process, we report controller CPU and memory, end-to-end detection latency, the control-channel rate

at which a single controller saturates, and the flow-rule aggregation and eviction policies required to keep a bounded TCAM from overflowing during a flood.

- **An evaluation-protocol audit.** We show that split methodology, not model capacity, accounts for most of the difference between the headline figures usually reported on this benchmark and the figures a chronological protocol yields, and we report both so the gap is visible. We pair this with a leakage audit, a collinearity and attribution-stability analysis, cross-vector and cross-day evaluation, and cross-dataset transfer to InSDN and CIC-IDS2017 with confidence intervals on the error rates.

We make no claim that XAI-SDN dominates prior work on every axis. The claim is narrower and, we think, more useful: an entropy-augmented tree ensemble with exact attributions occupies an operating point where detection quality, control-plane cost, and per-decision transparency can all be stated in measured numbers, and this paper states them, including where they are worse than the literature would lead one to expect. Two of those numbers concern the augmentation this system is named for, and they point in opposite directions. Under the chronological protocol the eighty exported flow statistics alone match the full 88-dimensional vector, so the entropy features do not improve in-distribution detection on this corpus. Under every shift for which we have data they do, provided all eight can be computed. Section VII-F reports both, together with the control that locates the cause of the first and the corpus that bounds the second.

Table 1 lists the symbols, abbreviations, and acronyms used throughout.

The remainder of the paper is organized as follows. Section II reviews related work. Section III defines the threat model and formalizes the detection problem. Section IV describes the XAI-SDN architecture and Section V its mathematical foundation. Section VI details the experimental setup. Section VII reports detection, latency, and explainability results. Section VIII presents the ablation and baseline comparison. Section IX discusses deployment considerations and limitations, and Section X concludes.

II. RELATED WORK

Work relevant to this paper falls into four groups. Entropy-based detectors established that distributional summaries expose flooding that per-flow statistics miss, but they act on fixed thresholds rather than learned boundaries. Learned detectors improved accuracy at the cost of transparency and, as several methodological critiques argue, of evaluation rigor. Explainable AI supplies the missing transparency, rarely with an accounting of what an explanation costs on the alert path. A fourth group, developed outside networking, builds nonlinear dynamic indices for abnormal-state detection and offers a vantage point on why distributional features work at all.

A. ENTROPY-BASED DDOS DETECTION IN SDN

Shannon entropy summarizes how concentrated a traffic attribute is over a window, and concentration is what a flood

TABLE 1. Notation, symbols, and acronyms.

Symbol	Meaning	Symbol	Meaning
f_i	i -th flow at the controller	W	sliding window of flow keys
$\mathbf{x}_i$	feature vector, $d = 88$	N	window capacity, in flows
$y_i, \hat{y}_i$	true, predicted label	n_k	multiplicity of key k in W
$g_\theta(\cdot)$	scoring function	$H(\cdot)$	Shannon entropy, in bits
s_i	DDoS score of f_i	S	running $\sum_k n_k \log_2 n_k$
τ	detection threshold	T	trees in the ensemble
$B(f_i)$	budget admission decision	L, D	max leaves, max depth
ϕ_i	exact TreeSHAP attribution computed for flow f_i	C	{Benign, DDoS}
ϕ_0	TreeSHAP base value	$\sigma(f_i)$	attack-episode signature
$\mathbf{a}_i$	attribution the alert record for f_i carries, Eq. (4)	$\rho(i)$	episode representative of f_i
TP, FP	true, false positive count	Acc	accuracy, Eq. (7)
		F1 _c	per-class F1
		TN, FN	true, false negative count
Acronym	Expansion	Acronym	Expansion
AP	average precision	ROC	receiver operating characteristic
AUC	area under the curve	SD-IoT	software-defined Internet of Things
DDoS	distributed denial of service	SDN	software-defined networking
FNR, FPR	false negative, false positive rate	SHAP	SHapley Additive exPlanations
IAT	inter-arrival time	SOC	security operations center
OXM	OpenFlow extensible match	TCAM	ternary content-addressable memory
PR	precision–recall	VIF	variance inflation factor
RF	Random Forest	XAI	explainable artificial intelligence

produces. Nychis et al. [12] characterized the correlation structure of entropy time series across several traffic distributions, and Lakhina et al. [13] showed that entropy exposes anomalies volume-based metrics do not register, establishing it as a standard anomaly-detection primitive [5]. In SDN, Mousavi and St-Hilaire [10] showed that a threshold on destination-address entropy, evaluated at the controller, detects flooding early enough to limit its impact, and Lall et al. [14] developed streaming estimators that make entropy tractable at line rate over sliding windows.

The relationship these methods exploit is statistical rather than deterministic. Low source-address entropy may indicate a concentrated flood, but the mapping depends on which attribute is summarized, how long the window is, what the background traffic happens to be, and how the attacker distributes its sources. An adversary who rotates spoofed addresses to hold the per-window distribution near uniform weakens the signal deliberately, a point Section IX returns to. Fixed thresholds also cannot represent interactions between distributional and flow-level evidence. XAI-SDN keeps the streaming entropy primitive but treats its output as features for a classifier rather than as a decision rule, at expected amortized $\mathcal{O}(1)$ cost per update under the assumptions stated in Section V.

B. MACHINE LEARNING FOR SDN DDOS DETECTION

Braga et al. [15] showed early that lightweight learning over OpenFlow counters suffices to detect flooding at the controller. Deeper architectures followed, including deep neural classifiers [16] and stacked-autoencoder detectors [17], trading inference cost and transparency for accuracy. Recent work

extends the progression into graph and sequence models: E-GraphSAGE [18] learns over the host-interaction graph, and FlowTransformer [19] applies encoder attention to flow sequences. Bhayo et al. [20] carry the approach into software-defined IoT, and Jain et al. [11] survey the field. Section VIII evaluates a representative of each family under one protocol on the same hardware, because published numbers are not comparable across papers.

Benchmarks have shaped the field as much as architectures have. CIC-IDS2017 [21] and CIC-DDoS2019 [4] supply labeled flow-level captures at a scale that supports supervised training, and InSDN [22] contributes traffic from an actual SDN testbed. Their convenience has a cost. Intrusion-detection surveys [23] and the critiques of Sommer and Paxson [24] and Arp et al. [25] document how easily results on such corpora are inflated: random splits scatter near-duplicate flows across both partitions, temporally segregated classes let a model key on capture phase instead of attack behavior, and synthetic generators leave per-feature signatures that separate the classes without any attack semantics being learned. Section VII reports what each of these contributes on the partition we evaluate.

C. EXPLAINABLE AI FOR INTRUSION DETECTION

Guidotti et al. [7] organize the space of black-box explanation methods, and Doshi-Velez and Kim [26] set out what rigorous evaluation of interpretability would require [6]. Two attribution methods dominate practice: LIME [27], which fits a local surrogate, and SHAP [28], which yields additive attributions with consistency guarantees. TreeSHAP [29] makes the latter exact and polynomial for tree ensembles, which is what

makes per-alert attribution viable in a real-time path. Gaspar et al. [30] apply both to a multilayer-perceptron intrusion detector, and Al and Sağiroğlu [9] survey the current state of the area. We use an exact method rather than a model-agnostic approximation because Warnecke et al. [8] find exact attributions both more faithful and cheaper in this setting.

Attribution answers which features drove a decision. It does not say how much confidence the decision deserves, and the two are often conflated. Swaminathan et al. [31] develop Bayesian machinery for quantifying predictive uncertainty, which is the complementary capability an operator needs when a score falls near the threshold. Our detector emits a vote fraction calibrated only in the loose sense of being monotone in evidence, and Section IX identifies Bayesian treatment of that score as the natural next step.

D. NONLINEAR DYNAMIC INDICES FOR ABNORMAL-STATE DETECTION

A parallel line of work, developed largely outside networking, argues that abnormality is easier to see in the structure of a distribution than in any individual observation. Chen et al. [32] construct a tensor Poincaré plot index for pumped storage units by decomposing an operational signal across time and frequency scales and extracting geometric indices from the resulting plots, on the argument that scale-agnostic summary statistics discard exactly what distinguishes a healthy machine from a degrading one.

The parallel to entropy-augmented flow detection is worth stating. Both replace a raw per-observation statistic with a distributional summary over a window, on the premise that the abnormal condition shows in the shape of the distribution rather than the magnitude of any single sample. Both must then answer whether the summary tracks the condition of interest or an incidental property of how the data were collected. Section VII answers that for our entropy features directly, by recomputing them over a shuffled arrival order and measuring how much discriminative power survives.

E. POSITIONING

Table 2 places XAI-SDN against the approaches above. What distinguishes this work is not the components, which are individually standard, but the combination of a streaming distributional feature set whose complexity and numerical behavior are both characterized, an explanation mechanism whose trigger is defined and whose cost is budgeted and measured, and an evaluation conducted at realistic class imbalance with the control-plane cost accounted for rather than assumed.

III. THREAT MODEL AND PROBLEM FORMULATION

We set out here what the defender observes, what the attacker controls, and how the detection task is formalized. We also fix four terms that are easy to conflate, because the distinction between classifying a flow and explaining that classification carries most of the computational cost reported in Section VII.

TABLE 2. Qualitative positioning. ✓ means the property is present and reported, × that it is absent or not reported. The columns are the axes on which this work differs: whether an attribution is produced for each decision, whether the cost of producing it is budgeted rather than incurred per positive prediction, whether control-plane cost is measured rather than assumed, and whether the evaluation protocol is audited.

Approach	XAI	Budget	Ctrl. cost	Audit
Mousavi and St-Hilaire [10]	intrinsic	n/a	×	×
Lall et al. [14]	intrinsic	n/a	×	×
Braga et al. [15]	×	n/a	×	×
Lo et al. [18]	×	n/a	×	×
Manocchio et al. [19]	×	n/a	×	×
Gaspar et al. [30]	post-hoc	×	×	×
XAI-SDN (this work)	exact	✓	✓	✓

A. THREAT MODEL

An attacker controls a botnet and launches a volumetric DDoS attack against an SDN-controlled network, flooding it with traffic and additionally generating packets that match no existing flow rule, so that table-miss events saturate the control channel and consume controller CPU. The attacker may spoof source addresses and vary packet timing but has no direct control over the controller or the switches. An adaptive attacker who optimizes against the detector is outside this model and is treated as a limitation in Section IX-D. The defender observes the flow statistics switches export and must decide in real time whether a flow is malicious, and produce a justification for that decision.

B. TERMINOLOGY

Four terms are used with fixed meanings throughout, including in the abstract, in Algorithm 1 and in every table reporting cost. A *flow* f_i is one bidirectional flow record exported to the controller; every flow is classified, so the number of classifications equals the number of flows. A *positive prediction* is a flow whose score reaches the detection threshold. An *alert* is a positive prediction dispatched to the security operations center: in the configuration evaluated here the two coincide numerically, but alerting is a policy decision layered on classification and the concepts are distinct. A *per-decision explanation* is an exact TreeSHAP attribution vector ϕ_i computed for one specific flow f_i , generated only for alerts the explanation budget admits, so per-decision explanations are in general a strict subset of alerts.

The fifth term is the one this distinction forces, and we state it separately because conflating it with the fourth would overstate what the system delivers. Alerts are grouped into *attack episodes* by a signature $\sigma(f_i)$, and the first alert of an episode that the budget admits becomes its *representative*, written $\rho(i)$ for the representative of f_i 's episode. An alert the budget declines carries the representative's attribution by reference rather than one of its own. Equation (4) writes this as $\mathbf{a}_i$, the attribution the alert record actually carries, and the two cases it distinguishes do not carry the same guarantee. Only the admitted case is an exact attribution of f_i 's own

score. The inherited case is exact for $f_{\rho(i)}$ and is an episode-level summary for f_i : it identifies the evidence that placed this episode over the threshold, not the evidence that placed this particular flow there. Section VII-K quantifies the trade and Section IX-D records the limit. We therefore never describe the system as computing an exact per-flow attribution for every alert; we describe it as computing one for every *admitted* alert, and as leaving every remaining alert resolvable to its episode representative.

Costs are reported against the quantity that drives them. Classification cost is per flow. Explanation cost is per generated explanation, never per flow and never per DDoS flow, because the number of explanations is a policy variable rather than a property of the traffic.

C. PROBLEM FORMULATION

Let f_i be the i -th flow observed at the controller and $\mathbf{x}_i \in \mathbb{R}^d$, $d = 88$, its feature representation: 80 CICFlowMeter statistics and 8 windowed entropy features. The detector learns a scoring function $g_\theta: \mathbb{R}^d \rightarrow [0, 1]$, writes $s_i = g_\theta(\mathbf{x}_i)$, and applies

$$\hat{y}_i = \mathbb{I}[s_i \geq \tau], \quad (1)$$

with τ selected on a validation partition, never on test data, and on the precision–recall plane rather than through the false negative rate, for the reason Section VII-H gives.

Explanation is a separate, conditional step. Writing $B(f_i) \in \{0, 1\}$ for the admission decision of the explanation budget, an explanation is generated exactly when

$$\text{explain}(f_i) = \mathbb{I}[\hat{y}_i = \text{DDoS}] \cdot \mathbb{I}[s_i \geq \tau] \cdot B(f_i). \quad (2)$$

Setting $B \equiv 1$ recovers the unbudgeted policy that an earlier version of this work assumed, and Section VII-K reports what it costs when almost all traffic is hostile. An explanation is an attribution vector $\phi_i \in \mathbb{R}^d$ satisfying

$$g_\theta(\mathbf{x}_i) = \phi_0 + \sum_{j=1}^d \phi_{ij}, \quad (3)$$

so the evidence decomposes exactly across features and can be audited after the fact.

Equation (3) is a property of an attribution computed for $\mathbf{x}_i$, and it is the reason a budgeted alert and an admitted one must be labeled differently. With $\rho(i)$ the representative of f_i 's episode as defined in Section III-B, the attribution the alert record carries is

$$\mathbf{a}_i = \begin{cases} \phi_i, & B(f_i) = 1, \\ \phi_{\rho(i)}, & B(f_i) = 0. \end{cases} \quad (4)$$

In the first case $\mathbf{a}_i$ satisfies (3) with respect to $\mathbf{x}_i$. In the second it satisfies (3) with respect to $\mathbf{x}_{\rho(i)}$, and it does so with respect to $\mathbf{x}_i$ only in the degenerate case $g_\theta(\mathbf{x}_i) = g_\theta(\mathbf{x}_{\rho(i)})$, which the budget does not enforce and which flows differing in duration, byte counts, ports or window composition will not in general satisfy. An inherited attribution is therefore an audit reference to an exact decomposition of the representative's

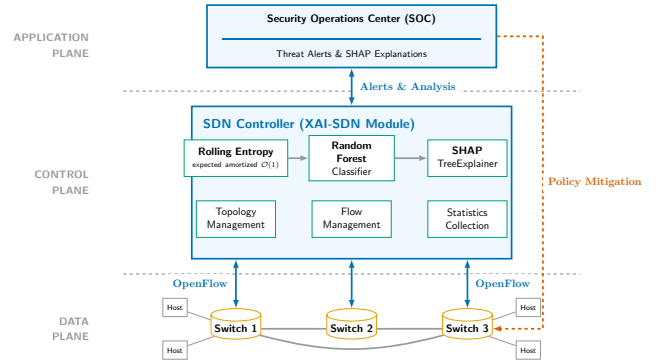


FIGURE 1. XAI-SDN system architecture. OpenFlow switches export flow statistics to the controller-embedded XAI-SDN module, which performs entropy-augmented feature extraction at expected amortized $\mathcal{O}(1)$ cost per update under the data-structure assumptions of Section V-C, Random Forest classification, and TreeSHAP attribution for the alerts the explanation budget admits. Detected DDoS events trigger alerts and policy updates; the dashed arrow represents the southbound policy-enforcement feedback loop.

score, not an exact decomposition of this flow's score, and no claim in this paper treats it as the latter.

The design objective is to maximize macro-averaged F1 under severe class imbalance, subject to a per-flow latency budget set by the control channel and an explanation budget set by the operator.

IV. XAI-SDN FRAMEWORK AND ARCHITECTURE

XAI-SDN comprises five stages: flow-statistics collection, feature extraction, Random Forest classification, explanation generation, and alert dispatch (Fig. 1).

An entropy engine (Fig. 2) maintains the distributional features of Section V from a circular buffer of recent flow keys and a hash map of key multiplicities, at expected amortized $\mathcal{O}(1)$ cost per update.

Positive predictions that the explanation budget admits are attributed with TreeSHAP [28], [29], and the resulting payload is forwarded to the security operations center for operator review. Algorithm 1 states the procedure, including the budget, which is the component that separates the cost of classification from the cost of explanation.

Two properties of Algorithm 1 are worth drawing out, because they determine the cost figures reported later. Classification occurs on line 7 for every flow without exception, so the classification rate equals the flow rate. Exact TreeSHAP attribution occurs on line 12 only when the budget admits the alert, so the rate at which attributions are *computed* is set by policy and not by the traffic mix. Every alert still leaves with an attribution attached, but not the same kind: an admitted alert carries its own exact vector, set on line 14, while a flow whose episode has already been explained carries a reference to the representative's vector, set on line 16. The audit trail is therefore complete in the sense of Eq. (4), every alert resolving to an attribution, and incomplete in the sense that only the admitted fraction resolves to an attribution of its own score. Setting $\mathcal{P}$ to admit unconditionally recovers the

Algorithm 1 XAI-SDN detection with a budgeted explanation trigger

Require: Flow batch $\mathcal{B} = \{f_1, \dots, f_n\}$; RF model $\mathcal{M}$; Tree-Explainer $\mathcal{E}$; window W ; threshold τ ; budget policy $\mathcal{P}$

Ensure: Label set $\mathcal{Y}$; alert set $\mathcal{A}$

```

1:  $\mathcal{Y} \leftarrow \emptyset, \mathcal{A} \leftarrow \emptyset, W \leftarrow \emptyset, \mathcal{P}.\text{Reset}()$ 
2: for each  $f_i \in \mathcal{B}$  do
3:    $\mathbf{x}_s \leftarrow \text{CICExtract}(f_i)$   $\triangleright$  80-dim statistics
4:    $\text{AdmitEvict}(W, f_i); \mathbf{x}_e \leftarrow \text{RollingEntropy}(W)$   $\triangleright$ 
   Eq. (11)
5:    $\mathbf{x} \leftarrow [\mathbf{x}_s; \mathbf{x}_e]$   $\triangleright$  88-dim vector, Eq. (12)
6:    $s_i \leftarrow \mathcal{M}.\text{Score}(\mathbf{x})$   $\triangleright$  Eq. (5)
7:    $\hat{y}_i \leftarrow \mathbb{I}[s_i \geq \tau]$   $\triangleright$  Eq. (1); every flow is classified
8:    $\mathcal{Y} \leftarrow \mathcal{Y} \cup \{\hat{y}_i\}$ 
9:   if  $\hat{y}_i = \text{DDoS}$  then  $\triangleright$  positive prediction
10:     $\sigma \leftarrow \text{Signature}(f_i)$   $\triangleright$  attack-episode identity
11:    if  $\mathcal{P}.\text{Admit}(\sigma, t_i)$  then  $\triangleright B(f_i) = 1$ ; Eq. (2)
12:       $\phi_i \leftarrow \mathcal{E}.\text{SHAPValues}(\mathbf{x})$   $\triangleright$  Eq. (6)
13:       $\mathcal{P}.\text{Record}(\sigma, \phi_i)$ 
14:       $\mathbf{a}_i \leftarrow \phi_i$   $\triangleright$  exact for  $f_i$ 
15:    else
16:       $\mathbf{a}_i \leftarrow \mathcal{P}.\text{Representative}(\sigma)$   $\triangleright \phi_{\rho(i)}$ , by
      reference
17:    end if
18:     $\mathcal{A} \leftarrow \mathcal{A} \cup \{\text{FormatAlert}(f_i, \hat{y}_i, s_i, \mathbf{a}_i)\}$ 
19:  end if
20: end for
21: return  $\mathcal{Y}, \mathcal{A}$ 

```

unbudgeted behavior in which the two rates coincide and every alert carries an exact per-flow attribution.

A. RANDOM FOREST CLASSIFICATION

Random Forest [33] is an ensemble of T trees $\{h_t(\cdot)\}_{t=1}^T$, each fitted to a bootstrap sample with a random feature subset at each split, and the label for $\mathbf{x}$ is the majority vote:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} \frac{1}{T} \sum_{t=1}^T \mathbb{I}[h_t(\mathbf{x}) = c], \quad (5)$$

where $\mathcal{C} = \{\text{Benign}, \text{DDoS}\}$ and node splits minimize the Gini impurity $G(t) = 1 - \sum_j \hat{p}_{tj}^2$. The fraction of votes for the DDoS class serves as the score $g_\theta(\mathbf{x})$ in Eq. (1).

B. SHAP EXPLAINABILITY

SHAP [28] computes the contribution ϕ_i of each feature i to a specific prediction $f(\mathbf{x})$ relative to the expected model output:

$$\phi_i = \sum_{S \subseteq \mathcal{F} \setminus \{i\}} \frac{|S|!(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} [f(S \cup \{i\}) - f(S)], \quad (6)$$

where $\mathcal{F}$ is the complete feature set. These values satisfy the additive efficiency property of Eq. (3), which is what makes an attribution auditable: the score decomposes exactly across features with no residual. TreeSHAP [29] computes

them exactly for tree ensembles in $\mathcal{O}(TLD^2)$ time, with L the maximum leaves and D the maximum depth per tree.

C. EVALUATION METRICS

Detection quality is reported from the confusion matrix, treating DDoS as the positive class. Writing TP, FP, TN, and FN for the four counts, Eq. (7) defines accuracy, the false positive rate, and the false negative rate:

$$\begin{aligned} \text{Acc} &= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \\ \text{FPR} &= \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad \text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}}. \end{aligned} \quad (7)$$

We also report per-class precision P_c and recall R_c , their harmonic mean $\text{F1}_c = 2P_cR_c/(P_c + R_c)$, and macro F1, the unweighted mean of the per-class F1_c , which is the accuracy-like summary that does not collapse to the majority class under severe imbalance.

Threshold-free performance is summarized by two curves rather than one. The area under the receiver operating characteristic curve (ROC-AUC) is reported for continuity with prior work, and the average precision (AP), the area under the precision-recall curve, is reported because it is the informative quantity at the class ratio evaluated here. The distinction is not cosmetic. A classifier drawing at random attains ROC-AUC 0.5 regardless of prevalence, but attains AP equal to the positive prevalence, which on our test partition is above 0.97. A ROC-AUC near unity is therefore close to uninformative on this data, whereas AP, precision, and the absolute count of false alarms are not. For the same reason, threshold selection in Section VII is performed on the precision-recall plane rather than on the false negative rate, which is normalized by the majority class and moves negligibly as individual errors are added or removed.

V. MATHEMATICAL FOUNDATION

The entropy features are specified below in full: what each one summarizes, which exported field it is computed from, how continuous quantities are discretized, how the sliding window is maintained, what the update recurrence is, and what that recurrence costs in both time and numerical accuracy. The level of detail is deliberate. An entropy feature is only reproducible if the window semantics and the binning are stated, and its complexity claim is only meaningful if the supporting data structures are named.

A. ENTROPY FEATURE EXTRACTION

Shannon entropy [34] measures how concentrated a discrete distribution is. Applied to a window of recent flows it summarizes the diversity of an attribute: source addresses, destination ports, protocol numbers, and so on. Concentration is often what a flood produces, so low entropy over a window may indicate an attack in progress and higher entropy is commonly associated with ordinary traffic. The relationship is statistical rather than deterministic, and it depends on the attribute chosen, the window length, the composition of the

TABLE 3. Specification of the eight entropy features: source field, transformation, and bin width.

Feature	Source field (exported)	Bin width
H_{src}	Source IP	categorical
H_{dst}	Destination IP	categorical
H_{dport}	Destination Port	integer, 1
H_{proto}	Protocol	integer, 1
H_{len}	Packet Length Mean	10 bytes
H_{iat}	Flow IAT Mean	1000 μs
H_{flag}	SYN Flag Count	integer, 1
H_{sport}	Source Port	integer, 1

background traffic, and the attacker's strategy. We treat entropy as evidence for a classifier to weigh, not as a decision rule.

Given a sliding window W holding the N most recent flows, the entropy of source addresses is

$$H_{\text{src}}(W) = - \sum_{i=1}^{|\mathcal{I}|} \frac{n_i}{N} \log_2 \frac{n_i}{N}, \quad (8)$$

where $\mathcal{I}$ is the set of distinct source addresses present in W and n_i is the number of flows in W carrying the i -th address. Equation (8) is evaluated identically for each of the eight attributes in Table 3; only the key changes. Continuous attributes are discretized by flooring to a fixed bin width before the entropy is taken, and the bin widths are given in the table rather than left implicit, since entropy over a continuous quantity is undefined without them.

Two points about this feature set deserve stating plainly. First, H_{len} and H_{iat} are computed over per-flow means, not over per-packet distributions, because the mean is what the flow exporter reports; they summarize the diversity of flow-level averages within a window and should be read that way. Second, H_{flag} summarizes the distribution of SYN flag counts rather than of a full TCP flag bitmask, for the same reason. An earlier version of this work listed a TTL entropy feature in place of H_{sport} . Flow records exported by CICFlowMeter contain no TTL field, so that feature was constant and carried no information; it has been replaced by source-port entropy, which is exported, is non-degenerate, and is directly relevant to floods that randomize source ports. The dimensionality of the representation is unchanged.

B. WINDOW MAINTENANCE AND THE UPDATE RECURRENCE

The window is a fixed-capacity circular buffer of flow keys paired with a hash map from key to multiplicity. Admitting flow f_i appends its key and increments that key's multiplicity; once the buffer is full, each admission also evicts the oldest key and decrements its multiplicity, deleting the entry when the count reaches zero. Before the buffer fills, entropy is computed over the $|W| < N$ flows seen so far, so the first N decisions of a cold start are taken on a partially populated window. In the offline evaluation each partition is processed

with a freshly initialized window, which prevents state from one partition influencing another.

Recomputing (8) from scratch on every admission would cost $\mathcal{O}(|\mathcal{I}|)$. Instead the engine maintains the scalar

$$S = \sum_{k \in \mathcal{K}(W)} n_k \log_2 n_k, \quad (9)$$

where $\mathcal{K}(W)$ is the set of distinct keys in W , and recovers the entropy as

$$H(W) = \log_2 |W| - \frac{S}{|W|}. \quad (10)$$

Admitting a key with current multiplicity n_a and evicting one with current multiplicity n_d updates S by

$$S' = S - \psi(n_a) + \psi(n_a+1) - \psi(n_d) + \psi(n_d-1), \quad (11)$$

with $\psi(m) \triangleq m \log_2 m$ and $\psi(0) \triangleq 0$.

Equations (9)–(11) are the recurrence the implementation executes. They are stated in this form deliberately: an update written directly on H , using terms of the form $-(m/N) \log_2(m/N)$, is only valid while $|W|$ is constant and therefore fails during window warm-up, whereas the S -form is correct throughout because the $|W|$ dependence is factored out into (10).

C. COMPLEXITY AND NUMERICAL BEHAVIOR

Each admit/evict pair performs a constant number of buffer operations, a constant number of hash-map operations, and a constant number of arithmetic operations, independent of N and of $|\mathcal{K}(W)|$. The circular buffer gives worst-case $\mathcal{O}(1)$ append and eviction. The hash map gives *expected* $\mathcal{O}(1)$ lookup, insertion, and deletion; under adversarial key collisions the worst case degrades to $\mathcal{O}(|\mathcal{K}(W)|)$ for an individual operation. The correct statement is therefore that the engine performs *expected amortized* $\mathcal{O}(1)$ work per update under these data-structure assumptions, not unconditional $\mathcal{O}(1)$. Where periodic exact resynchronization of S is enabled, as discussed below, it contributes a further $\mathcal{O}(|\mathcal{K}(W)|/R)$ amortized per update for a resynchronization interval of R . So that the bound can be checked against a measurement rather than taken on trust, the running-sum recurrence costs **381.5 ns** per admit/evict pair on the machine of Table 5, measured over the 1.2×10^7 -update stream of Section VII-I; the φ -form printed in the earlier version of this work costs 722.3 ns on the same stream.

Maintaining S incrementally accumulates floating-point error, since each update adds and subtracts terms of comparable magnitude. This is a legitimate concern for a detector intended to run continuously, and Section VII reports the measured drift over more than 10^7 updates, its effect on classifier decisions, and the cost of two mitigations: compensated summation, and periodic exact recomputation of S from the multiplicity map.

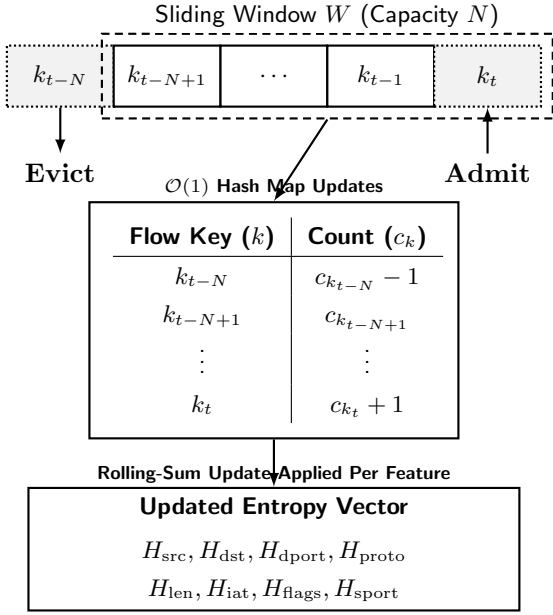


FIGURE 2. Rolling entropy engine. A circular buffer of the N most recent flow keys and a hash map of key multiplicities let each admit/evict pair update all eight entropy features with expected amortized $\mathcal{O}(1)$ work, independent of N . The update applied per feature is Eq. (11).

D. FEATURE VECTOR

The classifier input concatenates the 80 exported flow statistics with the eight entropy features of Table 3:

$$\mathbf{x} = [f_1, \dots, f_{80}, H_{\text{src}}, H_{\text{dst}}, H_{\text{dport}}, H_{\text{proto}}, H_{\text{len}}, H_{\text{iat}}, H_{\text{flag}}, H_{\text{sport}}], \quad (12)$$

where $\{f_j\}_{j=1}^{80}$ are the CICFlowMeter statistics and $\{H_k\}$ the eight entropy metrics, giving $d = 88$. Equation (12) is the vector scored by (1) and attributed by (6).

VI. EXPERIMENTAL SETUP

Four things are fixed before any result is reported: the corpora, the computing environment, the evaluation protocol, and the hyperparameter search. We state them so that every number reported later can be traced to a specific partition, a specific machine, and a specific selection procedure. Two matters that the earlier version of this work left implicit are made explicit here: how the train and test partitions are formed, and which observations were permitted to influence tuning.

A. DATASETS

The primary corpus is the SYN-flood partition of CIC-DDoS2019 [4], the 1.87 GB 03-11/Syn.csv capture. It holds 4,320,541 exported flow records. Rows carrying missing or infinite values are removed (283,076 rows), duplicate feature vectors are collapsed (446,671 rows), and labels are mapped to a binary target, leaving 3,590,794 flows of which 31,014 are benign. Flows are then ordered by export timestamp.

Generalization is assessed on five further corpora rather than on the SYN partition alone. Within CIC-DDoS2019 we

TABLE 4. Corpora used in this study, after cleaning.

Corpus	Role	Flows	Benign	Benign (%)
CIC-DDoS2019 SYN (03-11)	primary	3,590,794	31,014	0.86
CIC-DDoS2019 LDAP	vector	1,200,000	1,291	0.11
CIC-DDoS2019 NetBIOS	vector	1,200,000	827	0.07
CIC-DDoS2019 Portmap	vector	181,860	4,695	2.58
CIC-DDoS2019 SYN (01-12)	cross-day	1,200,000	356	0.03
InSDN	cross-set	223,372	68,419	30.63
CIC-IDS2017 DDoS	cross-set	223,082	95,068	42.62

use the LDAP, NetBIOS, and Portmap partitions of the same capture day to test transfer across attack vectors, and the SYN partition of the separate 01-12 capture day to test transfer across background conditions. Beyond CIC-DDoS2019 we use InSDN [22], collected on a real SDN testbed, and the DDoS and benign captures of CIC-IDS2017 [21]. Table 4 summarizes all of them.

Because these corpora were produced by different versions of the flow exporter, cross-corpus transfer is performed on the intersection of their schemas, computed explicitly and applied symmetrically so that the source model is trained on exactly the columns available at the target. Transfer to InSDN retains 79 flow statistics and all eight entropy features. CIC-IDS2017 exports no address, source-port, or protocol column, so four entropy features cannot be computed there and transfer retains 78 statistics and four entropy features. This is a property of the published corpora, not a modeling choice, and results are reported accordingly.

All corpora are publicly released and pre-anonymized. No additional personally identifiable information was collected or processed.

B. EVALUATION PROTOCOL

Split methodology turns out to matter more on this benchmark than any modeling decision we make, so the protocol is stated before any result.

The primary protocol is a *chronological* split: flows are ordered by export timestamp, the first 70% form the training partition and the last 30% the test partition, so that no training flow postdates any test flow. On the SYN partition this yields 2,513,556 training and 1,077,238 test flows, with 30,432 benign flows in the test partition, or 2.82%. Where a threshold or a hyperparameter must be selected, the last 10% of the training partition is held out chronologically as a validation block and the selection is made there; the test partition is read exactly once, at the reported operating point.

We also report, for comparison only, a *stratified random* split of the same proportions, which assigns 30% of each class to the test partition irrespective of time. This is the protocol implicitly used by much of the literature on this benchmark. Reporting both is not redundant. Section VII shows that the difference between them accounts for more of the gap between near-perfect and realistic performance figures than any architectural choice examined in this paper, and a reader

TABLE 5. Measurement environment and detector configuration. Every timing figure in this paper was produced on this machine, on the CPU. The tree depth is the scikit-learn default inherited from the submitted version; Section VII-Q reports what bounding it recovers.

Machine	Detector
CPU	Core i9-13900H, 14C/20T
Base clock	2.60 GHz
Cache	11.5 MB L2, 24 MB L3
Memory	47.6 GB
OS	Windows 11 Pro 26120
GPU	none used
Python	3.11.9
scikit-learn	1.8.0
NumPy / pandas	2.3.5 / 2.3.3
SHAP	0.51.0
XGBoost / LightGBM	3.2.0 / 4.7.0
PyTorch	2.14.0+cpu (no CUDA)
	Detector
	Trees T 200
	Max. depth none (fully grown)
	Features/split $\lfloor \sqrt{88} \rfloor = 9$
	Bootstrap enabled
	Class weight balanced
	Window N 1,000 flows
	Threshold τ 0.70
	Seed 42
	Stack
	OpenFlow os-ken, OF 1.3
	Emulator Mininet 2.3.0, OVS 3.9.1

comparing our numbers against published ones needs to know which convention produced each.

Hyperparameters are selected by nested cross-validation with forward-chaining outer folds, so that each fold trains on an expanding temporal prefix and tests on the block that follows it. The inner selection runs on a validation block carved from the end of each outer training prefix. The outer test block is never read during selection or fitting.

C. COMPUTING ENVIRONMENT

Every timing figure in this paper was measured on the single machine described in Table 5, on the CPU, with the same thread budget and the same measurement harness for every method. The installed PyTorch build carries no CUDA support, so no model in the comparison can be timed on a GPU even inadvertently. Latency and throughput figures across all methods and all tables are therefore directly comparable with one another. They are not comparable with figures published elsewhere on different hardware, and we make no such comparison.

D. IMPLEMENTATION, HYPERPARAMETERS, AND REPRODUCIBILITY

XAI-SDN is implemented in Python 3.10 using scikit-learn [35] and SHAP [28]. Random Forest hyperparameters are selected via grid search (Table 5); we use 200 fully-grown trees with the remaining settings as listed. All random seeds, artifact hashes, and configuration files are archived to support reproducibility.

VII. RESULTS

This section reports detection performance under the chronological protocol, then works through the questions that performance raises. We begin with the effect of split methodology, because it turns out to dominate, and because it determines how every subsequent number should be read. We then audit the data for leakage, quantify redundancy among the features and the stability of the attributions built on them, select the

TABLE 6. Effect of split methodology. Identical data, model, hyperparameters, and feature definition; only the partitioning differs. Both columns use the feature definition of the submitted version, including the defective TTL entropy feature of Section VII-C, so that the comparison isolates the split and nothing else. The headline figures of Section VII-B use the repaired feature set and differ from the chronological column here by 0.03 macro F1 points.

Metric	Stratified random	Chronological
Training flows	2,513,555	2,513,556
Test flows	1,077,239	1,077,238
Benign in test	9,304	30,432
Benign in test (%)	0.86	2.82
Accuracy (%)	99.9981	99.7538
Macro F1 (%)	99.9458	97.8467
ROC-AUC	1.000000	0.999721
False positive rate (%)	0.0000	0.1117
False negative rate (%)	0.0019	0.2501

operating threshold on the precision–recall plane, measure the numerical behavior of the streaming entropy engine, test generalization across attack vectors, capture days and corpora, and finally account for the cost of running the detector inside a live control plane.

A. SENSITIVITY TO THE EVALUATION PROTOCOL

The same model, the same features, and the same hyperparameters produce materially different headline figures depending only on how the data are partitioned (Table 6). We report this first because it determines how every subsequent number should be read.

Under a stratified random 70/30 split, the detector reaches 99.9981% accuracy and 99.9458% macro F1, with a false positive rate of zero on the test partition. Under a chronological 70/30 split of the same cleaned data, in which every training flow precedes every test flow, accuracy falls to 99.7538% and macro F1 to 97.8467%, while the false negative rate rises by more than two orders of magnitude.

The mechanism is visible in the class composition. A stratified split assigns exactly 30% of each class to the test partition, leaving 9,304 benign flows, or 0.86%. A chronological split leaves 30,432 benign flows, or 2.82%, because benign traffic in this capture is concentrated late in the recording rather than distributed evenly through it. A random split therefore disperses near-identical flows from the same short benign interval across both partitions, and the model is asked at test time about traffic it has already seen close analogues of.

We report this prominently for two reasons. It is the largest single effect we measure, larger than any architectural choice examined in this paper. It also explains a good deal of the near-perfect performance reported on this benchmark generally, and a reader comparing our figures against published ones needs to know which convention produced each. All results in the remainder of this paper use the chronological protocol unless stated otherwise.

TABLE 7. Detection performance on the chronological test partition, repaired 88-dimensional feature set, at the fixed $\tau = 0.70$ of Table 5 used throughout this paper. Section VII-H reports separately what selection on the validation block returns and why this corpus does not support it. Support is the number of test flows in each class. Per-class figures are computed from the confusion matrix of Section VII-M, and macro F1 is the unweighted mean of the two class F1 values shown.

Overall				
Accuracy			99.7579%	
Macro F1			97.8813%	
ROC-AUC / average precision			0.999728 / 0.999992	
False positive rate / false negative rate			0.0953% / 0.2464%	
Test flows (benign)			1,077,238 (30,432)	
Per class				
Class	Prec. (%)	Rec. (%)	F1 (%)	Support
Benign	92.1806	99.9047	95.8873	30,432
DDoS-SYN	99.9972	99.7536	99.8753	1,046,806

B. DETECTION PERFORMANCE UNDER THE CHRONOLOGICAL PROTOCOL

Table 7 reports the operating point of the full 88-dimensional detector on the chronological test partition of 1,077,238 flows, of which 30,432 are benign. Accuracy is 99.7579%, macro F1 is 97.8813%, ROC-AUC is 0.999728, and average precision is 0.999992. The false positive rate is 0.0953% and the false negative rate is 0.2464%.

The gap between ROC-AUC and macro F1 is the point made in Section VI in concrete form. At 97.2% positive prevalence a ROC-AUC of 0.9997 is close to uninformative, because the measure is insensitive to the minority class that the operator actually cares about. Macro F1 and the absolute false-alarm count are the figures that discriminate between models here, and they are what we report throughout. The lower half of Table 7 gives the per-class breakdown, where the asymmetry is plain: the majority class is detected almost perfectly while the benign class carries essentially all of the error.

C. FEATURE-SET AUDIT

Two properties of the feature set are worth reporting before any ablation, because both bear on how much of the representation is doing work.

Twelve of the 80 exported flow statistics are constant across all 3,590,794 flows of this partition and therefore carry no information: Bwd_PSH_Flags, Fwd_URG_Flags, Bwd_URG_Flags, FIN_Flag_Count, PSH_Flag_Count, ECE_Flag_Count, and the six bulk-rate fields. We note this in particular because FIN_Flag_Count was reported among the most influential features in an earlier version of this work, which cannot be correct for a column that never varies. The recomputed attributions in Section VII-G do not rank it.

Ranked by direction-agnostic single-feature AUC, the eight entropy features occupy the top eight positions, ahead of every exported flow statistic. The best flow statistic,

TABLE 8. Progressive removal of the most separable features, chronological protocol. Removal is in descending order of single-feature AUC.

Configuration	Dim.	Acc. (%)	Macro F1 (%)	FPR (%)
Full feature set	88	99.7579	97.8813	0.0953
Minus top 1	87	99.7679	97.9648	0.1446
Minus top 3	85	99.7590	97.8899	0.1347
Minus top 5 (all suspects)	83	99.7649	97.9394	0.1216
Minus top 10	78	99.8528	98.6911	0.0920
Minus top 20	68	99.8024	98.2580	0.0756

ACK_Flag_Count, reaches 0.8938 against 0.9986 for destination-address entropy. This supports the paper's central design choice and simultaneously raises a question that Section VII-D addresses directly: a windowed statistic can separate classes either because it captures attack structure or because it encodes position within a temporally segregated capture.

D. LEAKAGE AND DATASET-ARTIFACT AUDIT

Reviewers of an earlier version questioned whether performance on this benchmark reflects attack signal or dataset artifacts, and observed correctly that removing the three highest-ranked features tests very little. We therefore define the suspect set objectively rather than by intuition, as every feature whose single-feature AUC reaches 0.99, and remove the whole of it.

Five features meet that criterion, all of them entropy features: H_{dst} , H_{src} , H_{len} , H_{proto} , and H_{dport} . Table 8 reports performance as features are removed in order of separability.

Removing the whole suspect set leaves macro F1 at 97.9394%, marginally above the 97.8813% of the full model, and removing the ten most separable features raises it to 98.6911%. No small subset is load-bearing. We state the implication in both directions, because it cuts both ways: the result rules out a narrow leakage explanation, and it also shows that the discriminative information in this partition is distributed widely enough that feature-level ablation cannot by itself establish that the model has learned attack behavior.

No feature exhibits completely disjoint class ranges, which rules out the crudest form of generator artifact. After deduplication no test flow has a feature-identical twin in the training partition, and the median standardized distance from a test flow to its nearest training neighbor is 0.142, with no test flow closer than 10^{-6} .

Two controls bound the interpretation. Permuting the labels collapses the detector to chance, macro F1 0.4978 and ROC-AUC 0.5027, which is the result a pipeline with no label leakage must produce and which licenses the remaining numbers. Shuffling the row order before splitting restores macro F1 to 0.9995, reproducing the stratified figure and confirming that the gap in Table 6 is attributable to ordering alone.

Bootstrap resampling with 1,000 replicates puts the full model's macro F1 at 97.88% with a 95% interval of [97.81, 97.97] and the suspect-free model at 97.94% with

[97.86, 98.02]. The intervals overlap, so removing the flagged features produces no significant change. False positive rates are 0.096% [0.065, 0.130] and 0.122% [0.085, 0.162] respectively.

1) How much of the result was available to memorization

The census above is taken after deduplication, where it is zero by construction. What matters is what those duplicates would have done had they been kept, so we repeat the analysis on the raw partition. Before cleaning, 446,671 of 4,037,465 rows are exact duplicates, a rate of 11.06%, rising to 12.36% among benign flows. Whether that matters depends entirely on the split.

Under a stratified random 70/30 split, 15.49% of test rows have a feature-identical twin in the training partition, rising to 17.40% among benign flows. A lookup table that stores the training label of each distinct feature vector and predicts the majority class for anything unseen, learning nothing whatever, reaches 99.28% accuracy on that test partition. Under the chronological split the straddle rate falls to 0.036% overall and 0.003% for benign flows, and the same lookup table reaches 97.14%, which is simply the majority-class rate.

This does not show that the submitted model was a lookup table. It shows that on the stratified protocol 99.28 of the 99.9981 percentage points of accuracy were obtainable without learning anything about attacks, so the informative margin was roughly seven tenths of a point rather than the figure the headline suggests. On the chronological protocol the memorizable component collapses and the margin the model must earn is correspondingly larger.

E. WHY THE PROTOCOLS DIFFER: THE BENIGN CLASS IS NOT STATIONARY

The gap in Table 6 has a single cause, and it is a property of the capture rather than of any modeling decision.

Timestamps parse without error: zero of 4,320,541 records fail, the two candidate day-order conventions agree on every row, and the capture spans 6.1 hours. On that verified ordering, the median benign flow sits at position 0.995 of the timeline. Ninety-seven point nine percent of benign flows fall in the final 30% of the capture, and 97.3% fall in the final 10%. Attack traffic is spread throughout.

The consequence for chronological evaluation is severe. A 70/30 chronological split leaves 582 benign flows available for training against 30,432 at test, a ratio of one to fifty-two, and the proportion barely improves at other cut points (527 training benign at 50/50, 586 at 60/40). A chronological split of this partition is therefore not a harder version of the same task. It is close to a zero-shot problem for the benign class, and the 97.88% macro F1 of Table 7 should be read as a model generalizing to a class it has seen fewer than six hundred examples of.

This also explains the entropy result of Section VII-C. A windowed statistic computed over a capture whose classes are segregated in time partly encodes position within the capture, and position within the capture is almost perfectly predictive

of the label here. Section VII-F separates that effect from genuine attack structure.

We report this because it bears on every published result on this partition, not only ours. Any evaluation that splits CIC-DDoS2019 randomly distributes this narrow benign interval across both partitions and obtains a benign class that is easy by construction.

F. CONTRIBUTION OF THE ENTROPY AUGMENTATION

The entropy features are the component this work names itself after, so their contribution deserves to be measured rather than asserted. It was not measured in the submitted version, and when we measure it the result does not support the claim that was made.

Four feature configurations were fitted five times each, over the archived seeds {42, 123, 456, 789, 1024}, under the chronological protocol throughout. The *natural* configuration is the full 88-dimensional vector. The *CIC-only* configuration drops the eight entropy features. The *entropy-only* configuration keeps only those eight. The *scrambled* configuration is the control that matters most: the entropy features are computed over a random permutation of the arrival order and then returned to their chronological positions, so the features retain their marginal distributions but lose all temporal coherence, while the split and every other column stay identical. Table 10 reports the outcome.

Three findings follow, and we state the unwelcome one first. The entropy augmentation does not improve in-distribution detection on this corpus. The eighty exported flow statistics alone reach macro F1 98.28%, against 97.91% for the full 88-dimensional vector, and the difference favours the smaller feature set on every one of the five seeds. With five paired observations a Wilcoxon signed-rank test cannot return a p -value below 0.0625, which is what it returns, so the correct reading is that the effect is small, consistent in direction, and against us. The claim in the submitted manuscript that the entropy features improve accuracy is not supported and has been withdrawn.

Second, destroying the temporal coherence of the entropy features changes nothing measurable: 97.89% macro F1 scrambled against 97.91% natural, with a p -value of 0.4375 and the sign of the difference varying across seeds. Taken with Section VII-D, where the same scrambling drops each entropy feature's individual AUC from about 0.99 to about 0.56, this locates the effect precisely. The entropy features do carry genuine arrival-order information, and that information is almost entirely redundant with what CICFlowMeter has already encoded in the flow statistics. The classifier does not need it because it already has it.

Third, the entropy features cannot carry the task alone. Eight features give 49.31% macro F1 at a false positive rate of 99.89%, which is a model that labels almost everything an attack. That figure should not be read as a property of entropy features in general. Under the chronological protocol the training partition contains 582 benign flows, as Section VII-E

reports, and no eight-dimensional representation recovers a class from that.

1) Where the augmentation does earn its place

In distribution is the one setting in which CICFlowMeter has already encoded everything a distributional summary of the same window could add, so it is the setting least likely to reward the augmentation. A summary of how a traffic attribute is spread over a window is a claim about structure that ought to survive a change of attack vector, of capture day, or of network, where a raw per-flow counter such as a byte rate need not. We therefore repeated the comparison under every shift for which this paper already has data: three held-out attack vectors, a second capture day, and the two external corpora, each with three seeds. Table 9 reports it.

The result is conditional, and the condition is legible. Where all eight entropy features can be computed, the augmentation helps under shift and the per-seed ranges do not overlap: macro F1 rises from 0.6426 to 0.8364 on NetBIOS, from 0.4212 to 0.4820 across capture days, and from 0.2383 to 0.3811 on InSDN. On the two vectors where the flow statistics alone already exceed 0.96, it adds nothing measurable. Where it hurts, it hurts badly: on CIC-IDS2017 the full vector scores 0.2699 against 0.5658 for the flow statistics alone.

That last row is the informative one rather than an inconvenience. The CIC-IDS2017 export carries no address, protocol or source-port column, so four of the eight entropy features cannot be computed there and the “full” configuration on that corpus is a half-specified distributional summary rather than the augmentation this paper proposes. InSDN, which retains all eight, records the largest relative gain of any shift. The reading we take from the pair is that a partial entropy vector is worse than none, because the classifier allocates capacity to features that no longer mean what they meant in training.

We state the claim this licenses and no more. The entropy augmentation does not improve in-distribution detection on this corpus, and Section VII-K shows it is close to free. What it buys is robustness under distribution shift, in the cases where the full feature set is computable, and we have six shift conditions rather than a sample large enough to put a confidence interval around the effect. A reader should treat this as a directional finding consistent with what a distributional feature is supposed to do, not as a demonstration that it generalizes.

What the augmentation costs is the other half of the accounting, and it is small: the rolling engine adds expected amortized $\mathcal{O}(1)$ work per flow and a bounded buffer, which Section VII-I and Section VII-K quantify. A practitioner reading this section should conclude that on CIC-DDoS2019 the augmentation is close to free and close to useless, and that we have not established a setting in which it pays. Whether it pays where the exported statistics are poorer, on a corpus whose classes are not segregated in time, or under an adversary shaping per-window distributions, are the three questions this result raises and none of them are answered here.

TABLE 9. Macro F1 of the full 88-dimensional vector against the 80 flow statistics alone, in distribution and under every shift, mean over three seeds. “Entropy dims” is how many of the eight entropy features the target export allows to be computed. Bold marks a difference whose per-seed ranges do not overlap.

Target	Shift	Entropy dims	Flow stats	Full
SYN 03-11	none	8	0.9899	0.9670
LDAP	vector	8	0.9649	0.9666
NetBIOS	vector	8	0.6426	0.8364
Portmap	vector	8	0.9823	0.9855
SYN 01-12	capture day	8	0.4212	0.4820
InSDN	corpus	8	0.2383	0.3811
CIC-IDS2017	corpus	4	0.5658	0.2699

TABLE 10. Contribution of the entropy augmentation over five seeds under the chronological protocol (mean $\pm$ sample standard deviation). “Scrambled” recomputes the entropy features over a permuted arrival order with the split held fixed.

Configuration	Dim.	Acc. (%)	Macro F1 (%)	FPR (%)
Entropy only	8	96.893 $\pm$ 0.018	49.315 $\pm$ 0.004	99.886
Entropy scrambled	88	99.758 $\pm$ 0.004	97.886 $\pm$ 0.035	0.109
Full (natural)	88	99.761 $\pm$ 0.006	97.909 $\pm$ 0.051	0.120
Flow statistics only	80	99.805 $\pm$ 0.012	98.281 $\pm$ 0.102	0.095

G. COLLINEARITY, REDUNDANCY, AND ATTRIBUTION STABILITY

Reviewers of the earlier version asked what collinearity diagnostics were applied and whether the attributions built on the feature set are stable enough to be trusted operationally. Both questions have uncomfortable answers, and we report them.

The representation is severely collinear. Of 3,828 feature pairs, 61 have $|r| \geq 0.95$ and 19 have $|r| \geq 0.99$. Of the 76 non-constant features, **55 have a variance inflation factor above 10 and 32 above 100**, with a median VIF of 51.4, and the design matrix has a condition number of 3.9×10^{16} . By any conventional diagnostic the feature set is far past the point where individual coefficients would be interpretable in a linear model. Tree ensembles are not linear models and do not fail in that way, but the redundancy is real and it is what makes the removal results of Table 8 unsurprising in hindsight.

Reducing the feature set by correlation clustering costs nothing and returns latency (Table 11). Cutting at $1 - |\rho| = 0.05$ leaves 50 features and *raises* macro F1 to 97.99% while halving per-flow inference cost. Even 41 features hold 97.95%. Only at 29 features does performance fall below the full set.

Attribution stability is better than the collinearity would suggest, but only at the top of the ranking. Across ten seeds the five highest-attributed features are identical every time (top-5 Jaccard 1.00), and the full attribution vectors correlate at Spearman 0.956 on average, never below 0.932. Beyond the top five the ranking moves: top-10 Jaccard averages 0.710 and falls as low as 0.538. Across ten bootstrap resamples of the training data the picture is similar but weaker, with top-5

TABLE 11. Feature-set reduction by correlation clustering. Batch inference latency is per flow.

Feature set	Dim.	Acc. (%)	Macro F1 (%)	Lat. (ms)
Full	88	99.7579	97.8813	0.00219
Cluster cut 0.05	50	99.7706	97.9879	0.00113
Cluster cut 0.10	45	99.7670	97.9568	0.00136
Cluster cut 0.20	41	99.7660	97.9469	0.00154
Cluster cut 0.40	29	99.7198	97.5342	0.00145

Jaccard 0.767 and Spearman 0.940.

The operational reading is specific. An operator shown the top five contributing features is looking at a stable explanation. An analyst who reads significance into the difference between the eighth and twelfth ranked feature is reading noise. We state the limit rather than the headline, because the earlier version of this work inferred feature importance at exactly the depth where the ranking is not reproducible.

H. THRESHOLD SELECTION ON THE PRECISION-RECALL PLANE

Reviewer comment on the earlier version was that selecting τ from a sweep reporting “zero FNR below $\tau = 0.5$ ” is uninformative under severe imbalance, and that the precision-recall curve is the right instrument. That is correct, and acting on it exposed a further problem with this corpus.

Carving a validation block chronologically from the end of the training partition, as Section VI-B specifies, yields 2,154,476 training flows containing 522 benign, and 359,080 validation flows containing **60 benign**. Sixty negatives cannot support a stable threshold estimate: the validation average precision is 1.000000 and the validation ROC-AUC is 1.000000, both saturated, because the positive prevalence in that block is 0.9998. The selection procedure is sound and the data cannot support it, which is a direct consequence of the temporal structure documented in Section VII-E.

We report what the procedure returns and label it accordingly. Maximizing validation F1 on the fine-grained curve selects $\tau^* = 0.940$. At that operating point the chronological test partition gives precision 1.000000, recall 0.996330, zero false alarms out of 30,432 benign flows, and 3,842 missed attacks out of 1,046,806. Test average precision is 0.999990 against a chance baseline of 0.9717, and test ROC-AUC is 0.999665.

The threshold is therefore not tuned in any meaningful sense on this corpus, and we do not claim it is. What the analysis does establish is the methodological point the reviewer raised: at this prevalence the false negative rate moves by 10^{-6} per misclassified flow and conveys almost nothing, whereas precision and the absolute false-alarm count are normalized by the 30,432 negatives an operator would actually see. All threshold reporting in this paper uses the latter.

I. NUMERICAL STABILITY OF THE STREAMING ENTROPY ENGINE

An incremental update that repeatedly adds and subtracts terms of similar magnitude will accumulate floating-point error, and over a continuously running detector that error has no natural bound. The objection is well posed, so we measured it rather than argued about it.

A stream of 12,000,000 admit/evict updates over 65,524 distinct keys was driven through four implementations of the recurrence, with entropy recomputed exactly from the multiplicity map at 240 checkpoints. A rational-arithmetic evaluation at the midpoint confirms the float64 reference itself is exact to the printed precision, with an absolute difference of 0.

Worst-case drift across every variant is 2.3×10^{-13} bits. To convert that into the quantity that matters, all eight entropy features of all 1,077,238 test flows were perturbed by that magnitude and the detector rerun: **zero decisions changed**. Catastrophic cancellation is real in principle and, at this scale and in double precision, operationally absent.

Two incidental results are worth recording. Compensated summation and periodic resynchronization buy no accuracy here, so neither is enabled by default. And the φ -form printed in the earlier version of this work is both three times less accurate and nearly twice as slow as the running-sum recurrence the implementation actually executes, which is a further reason to state the latter.

J. TRANSFER: OTHER VECTORS, ANOTHER DAY, ANOTHER CORPUS

Training on one attack vector and testing on another is the first real test of whether the detector has learned attack structure or partition-specific detail. Table 12 reports three distances from the training distribution. The result separates cleanly into one positive finding and two negative ones.

Across attack vectors, transfer is strong. Trained only on SYN flooding and tested zero-shot on LDAP, NetBIOS and Portmap reflection attacks it has never seen, the detector reaches macro F1 0.969, 0.957 and 0.990, against 0.976 on its own held-out SYN traffic. Leave-one-vector-out gives the same picture from the other direction, at 0.966, 0.973 and 0.971 for the three reflection vectors. Transfer onto SYN from the other three is weaker, 0.697 at a false-negative rate of 6.9%. The representation carries across volumetric vectors, which is the evidence the SYN-only evaluation of the earlier version could not supply.

Across capture days it does not. Trained on the 03-11 SYN partition and tested on 01-12, macro F1 falls to 0.500. The false-negative rate stays at 3.5%, so the collapse is driven by the benign class, of which the 01-12 partition contains only 356 flows. A macro average over a class with 356 members is not a stable quantity, and we report the figure with that qualification rather than either suppressing it or treating it as a clean negative.

Across corpora it fails outright. On InSDN the detector reaches macro F1 0.603 and misses 57% of attacks. On CIC-

IDS2017 it reaches 0.279 and misses 98%, which is close to a detector that has stopped firing; the low false positive rate in that row is not a virtue, since a model that rarely alerts has few false alerts. Because the corpora come from different exporter versions, each condition runs on the intersection of their schemas, computed explicitly and applied symmetrically: InSDN retains all eight entropy features, while CIC-IDS2017 carries no address, protocol or source-port column and retains four. The targets themselves are not hard. Trained on them directly the same architecture reaches macro F1 0.999 on CIC-IDS2017 and 1.000 on InSDN, and joint training recovers the same figures, so the features and the model are adequate for both. What does not survive is the transfer.

Two caveats belong with these numbers. The InSDN export available to us is already binarized and mixes several attack families, so transfer to it measures general attack detection rather than DDoS detection, and part of the 57% miss rate is attributable to attack classes the detector was never trained on. Applying to InSDN the skepticism we applied to our own corpus in Section VII-D, an in-distribution macro F1 of exactly 1.000 after deduplication is implausibly perfect and most likely means InSDN is trivially separable under a random split. We have not audited that corpus to the depth we audited ours, and we present that row as evidence of feasibility and nothing more.

The honest summary is that this detector, trained on one corpus, should not be expected to work on another without target data. Section VII-J1 measures how much target data it takes, and Section IX-D restates the limitation.

1) What it costs to make the detector work on another network

Reporting a transfer failure and stopping leaves an operator with nothing to act on, so we measured the price of fixing it. Four conditions were compared at seven labelling budgets: the source model used zero-shot, a model trained on the target budget alone, the source forest warm-started and extended with target-fitted trees, and joint training on source data plus the budget. The train and test split of each target corpus stays chronological. Within the training partition the budget is drawn stratified rather than as a contiguous prefix, because on both corpora the earliest training flows are single-class, and because an operator labelling their own traffic samples across it rather than taking the first n records in timestamp order. Every condition here is evaluated on the held-out 30% of the target corpus rather than on all of it, and the source model is fitted on 600,000 flows rather than the full partition, so the zero-shot figures in this subsection are not the same numbers as in Table 12 and should not be read across. The comparison that matters is between conditions within the subsection, which share a test set.

On CIC-IDS2017 the answer is encouraging and it is not flattering to transfer learning. One hundred labeled target flows, of which 35 are benign, take macro F1 from 0.3547 zero-shot to 0.9964, and five hundred take it to 0.9990. But the model that achieves this is trained on the target budget

TABLE 12. Transfer at three distances from the training distribution: other attack vectors in the same capture, a second capture day, and two independent corpora. The source model is trained on the CIC-DDoS2019 03-11 SYN partition throughout. Bracketed dimensions are the shared schema width, which differs per corpus because not every entropy feature can be computed from every export. Leave-one-out results for the three reflection vectors, and the in-distribution and joint-training references for the two external corpora, are given in the text and in the released results.

Condition	Target	Macro F1	FPR	FNR
<i>Same capture, held-out attack vector</i>				
in-distribution	SYN 03-11	0.9759	0.0092	0.0075
zero-shot	LDAP	0.9692	0.0674	0.0001
zero-shot	NetBIOS	0.9566	0.0060	0.0001
zero-shot	Portmap	0.9896	0.0077	0.0009
leave-one-out	SYN 03-11	0.6966	0.0002	0.0693
<i>Second capture day</i>				
cross-day	SYN 01-12	0.4995	0.0112	0.0347
<i>Independent corpus, zero-shot</i>				
zero-shot	InSDN (87-dim)	0.6026	0.0010	0.5723
zero-shot	CIC-IDS2017 (82-dim)	0.2788	0.1737	0.9813

alone. Joint training matches it and warm-starting from the source forest is actively harmful, never exceeding 0.4613 at any budget and holding a false-negative rate above 0.91 throughout. The source model contributes nothing here, and an operator is better off discarding it and labelling a hundred flows than adapting it. We report that because it is what we measured, and because the opposite claim is the one a paper proposing a detector would prefer to make.

On InSDN no budget we tried is sufficient. The best result is 0.8513, from warm-starting at ten thousand flows, and the curve is not monotone: the same condition gives 0.7676 at five hundred flows and 0.7707 at fifty thousand. The reason is visible in the composition rather than the size of the budget. InSDN's chronological training partition holds 1,499 benign flows out of 156,360, so a stratified draw of fifty thousand contains 479 benign examples and a draw of a hundred contains five. What an operator has to supply is not n labeled flows but n labeled examples of the minority class, and on that corpus they are not available at any budget. This is the same non-stationary benign class that Section VII-E documents for our own partition, and it is a property of how these captures are built rather than of the detector.

K. THE COST OF EXPLANATION, AND HOW TO BOUND IT

Reviewer 6 objected that Algorithm 1 will trigger an attribution for nearly every flow when almost all traffic is hostile, and that the resulting rate will bottleneck the controller. The objection is correct. This section measures the collapse and then measures three ways of preventing it.

The two stages differ by a factor of fifty. Over a 60,000-flow stream at 99.998% attack prevalence, classification costs 0.028 ms per flow, sustaining 35,715 flows/s, while one Tree-SHAP attribution costs 1.485 ms, sustaining 673 explanations per second. The alert rate on this stream is 0.99997, so the unbudgeted policy explains essentially every flow and the

sustained rate falls to **661 flows/s**, a factor of 54 below the classification rate. Table 13 reports what each budget policy recovers.

The mechanism that works is not a faster explainer but a trigger that distinguishes an alert from an episode. Structurally identical flood flows are one attack, and the alert record for each can carry a reference to the episode representative, $\phi_{\rho(i)}$ in Eq. (4), rather than an attribution computed for itself, so every alert still resolves to an attribution while the number of TreeSHAP evaluations falls. How much it falls depends entirely on how an episode is defined, which is the finding we did not expect. A signature that includes per-flow volume terms fragments the flood into 53,817 distinct episodes and deduplicates almost nothing, giving a $1.1\times$ reduction. A signature on the target address and protocol collapses the same stream to two episodes, a reduction of 3.0×10^4 , and restores **35,652 flows/s**, which is the classification rate to within 0.2%.

The cost of that reduction is stated rather than hidden, because it is not free. A coarse episode explains why a flood against a given service is being blocked. It does not explain why one particular flow within the flood was scored the way it was, and an operator investigating a suspected false positive needs the latter. The policy is therefore a scale control, to be tightened as alert volume falls, and the middle rows of Table 13 are where a deployment that wants per-target rather than per-service explanations would sit. The token-bucket policy makes the opposite trade, keeping full per-flow fidelity for a fixed number of alerts per second and none for the rest: at 50 explanations per second it sustains 33,273 flows/s with 0.14% of alerts individually attributed. Asynchronous off-path attribution is the least useful of the three here, because a queue large enough to avoid drops simply moves the bottleneck without relieving it, sustaining 669 flows/s at a 1,024-entry queue, while a 128-entry queue reaches 1,314 flows/s only by dropping 51.2% of the attributions.

L. GLOBAL FEATURE ATTRIBUTION

Averaging the absolute TreeSHAP value of each feature over 3,000 test flows and across ten seeds gives the ranking shown in Fig. 3. Three observations follow, and the first two correct the earlier version of this work.

The single most influential feature is the ACK flag count, not a volumetric statistic. Source and destination address entropy follow immediately, and together those three account for most of the attributed signal. The earlier version reported `Flow_Bytes/s`, `Destination_Port` and `FIN_Flag_Count` as the leading features. The first two do appear in the ranking, at positions six and seven, but `FIN_Flag_Count` is constant across every flow in this partition (Section VII-C) and cannot contribute to any prediction. It does not appear in the recomputed ranking.

Three of the top twelve features are entropy metrics, not six as previously stated. The entropy features are influential, and the two address entropies sit at ranks two and three, but they do not dominate the attribution in the way the single-feature

TABLE 13. Explanation budget policies over a 60,000-flow stream at 99.998% attack prevalence, all rows from one run of the same experiment. “Explanations” counts exact TreeSHAP evaluations, and “Coverage” is that count divided by the alerts eligible under the policy, so it is the fraction of alerts receiving an exact attribution of their own score, $a_i = \phi_i$ in Eq. (4). Under episode deduplication every remaining alert carries its representative’s attribution $\phi_{\rho(i)}$ by reference: those alerts resolve to an attribution and are not left unexplained, but that attribution is exact for the representative rather than for them. * the P3 rows are measured on a 20,000-flow prefix of the same stream, so their coverage denominator is 20,000 alerts rather than 59,998.

Policy	Explanations	Coverage	Flows/s
Classification only (no attribution)	0	n/a	35,715
P0 attribute every alert	59,998	1.000	661
P1 episode, per-flow signature	53,817	0.897	735
P1 episode, per-target signature	38,046	0.634	1,031
P1 episode, per-service signature	2	0.000	35,652
P2 token bucket, 50/s	83	0.001	33,273
P2 token bucket, 500/s	839	0.014	20,505
P3 asynchronous, 128-entry queue*	9,760	0.488	1,314
P3 asynchronous, 1,024-entry queue*	20,000	1.000	669

separability reported in Section VII-C might suggest. The reconciliation is that the classifier has many correlated routes to the same information, so no individual feature receives a large share of the credit.

Only the top five entries are reproducible across seeds (Section VII-G). Ranks below that shift between runs, because from rank six downward the seed-to-seed standard deviation of the mean absolute attribution is comparable to the gaps between adjacent features. We therefore draw conclusions from the top of the ranking only, and the full ranking with its per-seed deviations is in the released results.

Fig. 3 illustrates the same attributions per flow, and provides an unplanned demonstration of that caveat. Its sample over-represents benign flows so that both classes are visible, and under that sample the leading three features are the same three, in a different order: source-address entropy first at 0.0727 mean absolute attribution, the ACK flag count second at 0.0638, destination-address entropy third at 0.0629. Membership of the top group is stable and the ordering within it is not, which is exactly what Section VII-G measures and what an operator reading an attribution should be told.

M. INDIVIDUAL ALERT CASE STUDIES

Aggregate attribution statistics say which features matter on average. They do not show whether an explanation is useful to the person who has to act on one alert at three in the morning, which is the claim the mechanism has to earn. Table 14 therefore reports four flows drawn from actual test predictions: the highest-scoring true positive, the highest-scoring false positive, the lowest-scoring false negative, and the flow whose score sits closest to the threshold. On the chronological test partition the detector produces 1,044,227 true positives, 30,403 true negatives, **29 false positives** and 2,579 false negatives, so the false positive shown is one of a very small population.

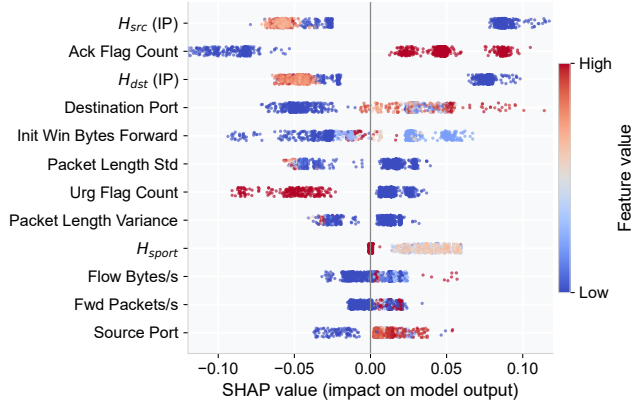


FIGURE 3. Per-flow TreeSHAP attributions for the twelve highest-ranked features, 1,500 test flows under the chronological protocol. Each point is one flow, positioned by its signed contribution to the DDoS score and colored by that feature's value. Benign flows are over-represented in this sample, at 35% rather than the 2.8% of the test partition, because a sample drawn at the natural ratio shows one class only; no number in the text is computed from this sample.

TABLE 14. Four alerts drawn from actual test predictions, each shown with the three features contributing most to its score and the signed TreeSHAP value of each. Scores are the forest vote fraction; the threshold is 0.70.

Case	Score	Three largest contributions
True positive	1.000	$H_{src}=0.00$ (+0.088); $H_{dst}=0.00$ (+0.080); ACK count=1 (+0.047)
False positive	0.780	ACK count=0 (−0.083); dst port=53 (−0.061); bytes/s= 5.8×10^7 (+0.057)
False negative	0.125	ACK count=0 (−0.091); $H_{src}=2.74$ (−0.072); $H_{dst}=4.26$ (−0.064)
Borderline	0.700	ACK count=0 (−0.050); $H_{src}=3.59$ (−0.042); duration= 1.12×10^8 (+0.034)

Read together the four cases describe one coherent mechanism. The true positive carries source and destination entropy of exactly zero, meaning every flow in the window shares one address pair, and those two features supply the largest positive contributions. The false negative is its mirror image: an attack flow whose window happens to show source entropy of 2.74 and destination entropy of 4.26, and both features push the score down hard enough to keep it below the threshold. The borderline case sits between them with the same features pulling in the same direction but less strongly.

The false positive is the case that justifies the expense of per-decision attribution. It is a port 53 flow carrying 58 Mbit/s, and the attribution shows the model was not persuaded by the entropy evidence at all. Both the ACK flag count and the destination port push the score down. What carried it over the threshold was the byte rate. An analyst reading that attribution can dismiss the alert on the grounds that a high-rate DNS flow is expected on this network, and can point at the specific feature responsible if the rule needs tuning. Without it the analyst sees only a score of 0.78 and has nothing to act on.

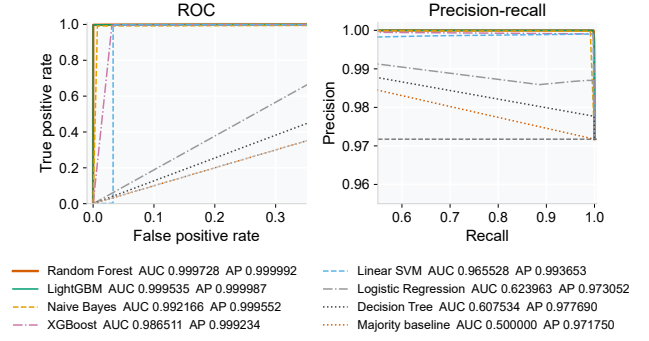


FIGURE 4. ROC and precision-recall curves on the full chronological test partition, 1,077,238 flows at 97.17% positive prevalence, every model measured by us on the same data. Axes are zoomed to the region where the curves differ. The majority baseline gives the chance operating point in both panels: ROC-AUC 0.5000 and average precision 0.9717. Legend entries are ordered by ROC-AUC.

N. ROC AND PRECISION-RECALL CURVES

The submitted version discussed ROC curves for the baselines without presenting them. Fig. 4 supplies them for the full chronological protocol, paired with precision-recall curves for the same models, and the pairing is the point rather than a convention.

At 97.17% positive prevalence a classifier that predicts at random attains average precision equal to that prevalence. The majority baseline in the figure therefore sits at AP 0.9717 while its ROC-AUC is 0.5000, and the gap between a model's two numbers is the honest measure of how much the imbalance flatters its ROC curve. Two rows make the point sharply. The decision tree reaches AP 0.9777 against a chance value of 0.9717, an improvement of six thousandths, while its ROC-AUC of 0.6075 reads as though it were doing something. Logistic regression is worse still: AP 0.9731 is within 0.0014 of chance. Neither model is usable, and only the precision-recall panel says so.

The detector reaches ROC-AUC 0.999728 with a bootstrap interval of [0.99970, 0.99975] over 150 resamples, and average precision 0.999992. Its nearest competitor on ranking quality is LightGBM at 0.99953, and the two are separated far more clearly by their false-alarm counts than by either curve.

Curves rank; they do not choose an operating point, and the two can diverge badly. We therefore also report each model at a threshold selected on the first half of its test scores and evaluated on the second, alongside the fixed $\tau = 0.70$ used throughout the paper. For the tree ensembles the two agree to within two points. For the graph model of Section VIII-B they do not: its ROC-AUC is 0.9973, yet at any threshold selected this way it classifies almost nothing as an attack, because its outputs are compressed into a narrow band well below τ . Reporting only a curve would present that model as competitive with ours. It is competitive at ranking and unusable as a detector, and the distinction matters to anyone who has to deploy one.

O. HYPERPARAMETER SENSITIVITY

Two hyperparameters govern the detector: the detection threshold and the entropy window length. They are treated differently here, and the difference matters for the tuning and testing separation that Section VI-B describes.

The threshold is selected, and Section VII-H reports that selection in full, on the validation partition alone, together with the reason this corpus does not really support it. We do not repeat the sweep here.

The window length was not tuned. It was fixed at $N = 1000$ flows before any evaluation and held there for every experiment in this paper. That is a deliberate choice rather than an omission. With only 60 benign flows in the validation block, a window length selected on that block would be fitted to noise, and selecting it on the test partition would invalidate every number that follows. A sensitivity sweep over N under the corrected protocol is work we have not done, and we do not carry over the sweep reported in the earlier version of this manuscript, which was measured under the stratified split and is not comparable to anything reported here.

P. LIVE DEPLOYMENT ON AN EMULATED NETWORK

The testbed of Section VII-R is a control plane without a data plane: it measures what the controller costs, but no packet ever traverses a switch. Reviewer 4 asked for a proof of concept with live traffic, and the question it answers is a different one, namely whether the mitigation the detector installs actually protects anything.

The topology is one Open vSwitch datapath speaking OpenFlow 1.3 to the detector running inside a controller process, with three hosts on 100 Mbit links at 1 ms delay: an attacker, a benign client, and a server. The benign client runs an iperf3 transfer and a ping series against the server, first with the network idle and then while the attacker runs a randomized-source SYN flood from hping3. Three conditions are compared. Detection is applied to TCP only, because the detector is trained on TCP flow records and scoring an ICMP packet would evaluate the model on a vector whose TCP fields are all zero.

Table 15 reports the result and Fig. 5 shows the two comparisons that carry it. The first row is the one that matters. With no detector the flood costs the benign client 73% of its throughput, falling from 95.39 to 25.78 Mbit/s. With the detector the same flood costs it nothing measurable: 95.29 to 95.39 Mbit/s, zero added packet loss, and a ping round-trip time that does not rise. The switch holds 997 drop rules against distinct spoofed sources by the end of the attack, against four flow entries and no drop rules in the undefended condition.

Two further observations come out of the same run. Mitigation pays for its own control-plane cost: packet-in events reaching the controller fall from 40,160 undefended to 2,335 with detection enabled, because the installed rules stop the flood at the switch, so the controller stops seeing it. And attributing every admitted alert costs 21% more decision latency, 10.84 ms against 13.12 ms at the median, and 34%

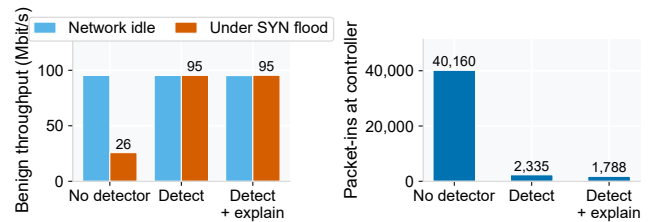


FIGURE 5. The emulated network under a randomized-source SYN flood. Left: benign iperf3 throughput with the network idle and during the flood, for each of the three conditions. Right: packet-in events reaching the controller over the same run. Installing drop rules at the switch both preserves the benign transfer and removes most of the control-plane load the flood would otherwise impose.

TABLE 15. Emulated network under a randomized-source SYN flood. Throughput is a 100 Mbit/s iperf3 transfer by the benign client; the flow table is read at the end of the attack. The undefended row uses the same learning-switch controller with detection disabled, so the comparison isolates the detector rather than the presence of a controller.

	No detector	Detect	Detect + explain
Throughput idle (Mbit/s)	95.39	95.29	95.20
Throughput under attack	25.78	95.39	95.48
Fraction retained	0.270	1.001	1.003
Added packet loss (%)	0.0	0.0	0.0
Flow entries / drop rules	4 / 0	999 / 997	621 / 619
Packet-ins at controller	40,160	2,335	1,788
Controller CPU (% of a core)	32.5	101.0	100.9
Controller memory (MB)	58	190	255
Decision latency p50 (ms)	n/a	10.84	13.12

more resident memory, with no measurable difference in delivered quality of service. That is the same trade Section VII-K quantifies offline, now visible on a data path.

This is a three-host emulation and we do not present it as more. It cannot exhibit multi-datapath rule distribution, controller failover, or the traffic patterns of an operational fabric, and Section IX-D says so. What it does establish is that the mitigation path works end to end against live traffic, which no offline measurement in this paper can.

Q. MODEL SIZE, LATENCY AND WHAT THE CONFIGURATION ACTUALLY COSTS

The detector inherits its configuration from the submitted work, where it was never chosen: 200 trees grown to unbounded depth are the library defaults. With a single-flow decision costing 7.05 ms against 0.26 ms for LightGBM at the same macro F1, the obvious hypothesis is that the trees are enormous and that bounding them would recover most of the gap. We swept depth and ensemble size jointly to test it. The hypothesis is wrong, and the way in which it is wrong is the useful part.

The trees are not enormous. Across the whole grid, including unbounded depth, they average between 32.7 and 46.0 leaves, and the largest forest serializes to 1.46 MB. The cause is the class balance rather than the depth limit: with 533 benign flows in a 2,262,199-flow training partition and class-

balanced weighting, splitting stops early for want of minority examples, so an unbounded tree and a depth-8 tree end up nearly the same size. A reader who assumed, as we did, that the default configuration was pathological would have drawn the wrong conclusion from the latency figure.

Detection quality is equally insensitive. Table 16 reports the frontier, one row per ensemble size at three depths. Macro F1 across all 24 configurations spans 0.9778 to 0.9799 and the false positive rate spans 0.09% to 0.14%. Neither depth nor ensemble size is load-bearing on this corpus, which is consistent with Section VII-D: the discriminative information is spread widely enough that the model has many routes to it.

What does scale is memory, and with it latency. The ensemble size sets the serialized footprint almost linearly, from 0.14 MB at 25 trees to 1.47 MB at 200. Taking the smallest configuration whose macro F1 is within 0.005 of the best gives depth 8 with 25 trees: a tenfold smaller model for a macro F1 change of -0.0009 and a false positive rate 0.02 percentage points higher. On a controller, or on the constrained gateway of Section IX-B, that is worth taking on memory grounds alone, and it turns out to buy latency too.

Latency needs a measurement the sweep cannot give. Each configuration in the sweep is timed immediately after being fitted, while the fitting thread pool is still winding down, and the resulting figures are non-monotonic in model size and differ by a factor of two between neighboring configurations. We therefore fitted only the two endpoint configurations in a single process and probed them alternately over five rounds, so that any machine state affects both equally.

That gives 6.30 ms for the inherited configuration against 0.89 ms for the selected one, repeatable to within 5% across rounds: a $7.0\times$ reduction for the -0.0009 macro F1 already noted. Latency is therefore close to linear in ensemble size, at 0.032 ms per tree for the 200-tree forest and 0.035 ms per tree for the 25-tree one, and almost independent of the depth limit, which is what the uniform 46-leaf tree size predicts. The order-of-magnitude gap against LightGBM in Table 19 is a per-tree property of the prediction path rather than anything a change of ensemble size closes. Section VII-K reports the batched rate of 35,715 flows/s, which is what the detector achieves when the per-call cost is amortized across a batch.

One caveat applies to the selection. It was meant to be made on the validation block, but that block holds 49 benign flows out of 251,356 and every configuration scores identically on it, so validation cannot separate them. The choice above is therefore a reading of the frontier rather than a protocol-clean selection, and we label it as one. The corpus does not support model selection here for the same reason it does not support threshold selection (Section VII-H).

R. CONTROL-PLANE COST AND THE RATE A CONTROLLER CAN INGEST

Every throughput figure so far is an offline one: features are read from a matrix and scored in a loop. Reviewer 3 asked how 60,000 novel flows per second are physically ingested without saturating the OpenFlow control channel, and that

TABLE 16. Model-size frontier, chronological protocol, evaluated on all 1,077,238 test flows. One row per ensemble size at three depths; the full grid of six depths is in the released results. Macro F1 spans 0.002 across every configuration measured.

Depth	Trees	Size (MB)	Leaves/tree	Macro F1	FPR (%)
8	25	0.14	35.2	0.9782	0.12
8	200	1.06	32.7	0.9778	0.10
16	25	0.18	44.8	0.9799	0.14
16	200	1.44	45.0	0.9790	0.10
unbounded	25	0.18	45.3	0.9799	0.12
unbounded	200	1.46	46.0	0.9791	0.10

question cannot be answered offline. We therefore run the detector inside a controller process that exchanges genuine OpenFlow 1.3 messages with switch agents over TCP, with the encodings validated against the os-ken reference parser, and measure the controller itself rather than the model. Each session binds its own port, so a controller left behind by an earlier session cannot absorb the next one's traffic.

At an offered rate of 3,000 packet-in events per second the controller saturates. It sustains 114 events per second end to end, including classification and rule installation, at 98.3% of one core and 176 MB resident, with a decision taking 6.61 ms at the median and 32.70 ms at the 99th percentile. Turning on attribution for every alert halves the sustained rate to 61 events per second, nearly doubles the median decision to 11.33 ms, and raises resident memory to 234 MB. Benign background traffic through the same controller pays for this: its 95th-percentile round-trip time rises from 34.11 ms to 41.16 ms, an inflation of $1.21\times$. That is the quality-of-service cost of the security function, measured on unrelated flows rather than assumed to be negligible.

Sweeping the offered rate locates the knee and shows what lies past it (Table 17). The controller reaches its maximum of 77 events per second at 500 offered. Above that, goodput does not plateau but collapses to roughly 35 events per second and stays there, while the median round-trip time nearly triples. Offering more work returns less, which is the behavior of a queue past saturation and not a gradual degradation an operator would notice from a throughput number alone.

The answer to the reviewer's question follows directly and it is not the answer the submitted manuscript implied. A single controller instance on this hardware ingests at most 77 novel flows per second as individual table-miss events. The 60,606 flows per second reported earlier was an offline feature-extraction and inference rate measured with no control channel present, and it is not an ingest rate. Reaching that order of magnitude at the control plane requires that the flows not arrive as individual packet-in events at all: switch-side aggregation, packet-in rate limiting, and sampled export are the mechanisms, and Section VII-S quantifies what the first of them costs in flow-table occupancy. At the observed 386 bytes per packet-in, 60,000 events per second would also require 23 MB/s on the control channel, which is a separate constraint from controller CPU and one that no aggregation

TABLE 17. Offered-rate sweep on the OpenFlow testbed, detection without attribution. “Achieved” is the end-to-end rate including classification and rule installation. Goodput peaks at an offered rate of 500 and collapses above it. The 2,000, 8,000 and 16,000 rows lie on the same plateau and are in the released results.

Offered (/s)	Achieved (/s)	Ratio	RTT p50 (ms)	RTT p99 (ms)	CPU (%)
500	77	0.155	10.20	38.00	92
1,000	34	0.034	27.46	49.17	89
4,000	34	0.009	27.59	57.12	88
32,000	35	0.001	26.92	48.68	91

policy removes.

S. FLOW-TABLE OCCUPANCY UNDER MITIGATION

Reviewer 3 asks what flow-rule aggregation and eviction keep a bounded TCAM from overflowing during a flood. Answering it required first checking the premise the question rests on, which is that a spoofed flood installs one table entry per spoofed source. On this corpus that premise does not hold, and the measurement that matters is a different one.

Across the whole SYN partition, 99.08% of the 3,590,794 flows carry a single source address. In the 40,000-flow window replayed through the testbed there are three distinct source addresses, occupying two /24 networks, and three distinct destination addresses. The flood is not source-diverse at the flow-export level. What is diverse is the port field: the same window contains 29,064 distinct destination ports and 34,624 distinct source ports, which is what makes 29,065 distinct exact-match keys out of 40,000 flows.

Table 18 reports rule installations, peak occupancy, and overflows for six aggregation policies at three table capacities. The consequence of the above is immediate. Exact-match installation saturates the table and overflows continuously, which is the failure mode the reviewer identifies. Collapsing the source address to a /24 or a /16 changes nothing, to four significant figures, because there is no source diversity to collapse. Wildcarding the source entirely changes nothing either, for the same reason. The aggregation that bounds the table is the one that wildcards the port: a rule matched on source address, destination address, and protocol occupies three entries for the whole window, installs 0.07 rules per thousand flows against 913.3 for exact matching, a reduction of 1.2×10^4 , and never overflows at any capacity tested. Choice of eviction policy moves the installation count by less than 0.2% under every aggregation, so it is second-order here; the table reports least-recently-used and the full grid is in the released results.

Two qualifications belong with that result. The first is that the reduction is bought with specificity: a rule matched on destination address and protocol drops legitimate traffic to the same host, so port-wildcarded mitigation is an instrument of last resort rather than a default, and the decision of when to escalate to it is a policy question the detector informs but does not settle. The second is that the source-spoofed case,

TABLE 18. Flow-table occupancy over a 40,000-flow window offered at 3,000 flows/s, under least-recently-used eviction. “Inst./1k” is rule installations per thousand flows; the match is given as the fields retained, everything else being wildcarded. The port-wildcarded policies behave identically at every capacity, so one is shown. The full grid of six aggregations, four eviction policies and three capacities is in the released results.

Capacity	Match retained	Inst./1k	Peak	Overflows	Hit rate
1,024	src, dst, dport, proto	955.6	1,024	37,198	0.044
1,024	src/24, dst, dport, proto	955.6	1,024	37,198	0.044
1,024	src/16, dst, dport, proto	955.6	1,024	37,198	0.044
1,024	dst, dport, proto	955.6	1,024	37,198	0.044
1,024	src, dst, proto	0.07	3	0	1.000
1,024	dst, proto	0.07	3	0	1.000
4,096	src, dst, dport, proto	913.3	4,096	32,436	0.087
4,096	src/24, dst, dport, proto	913.3	4,096	32,436	0.087
4,096	src/16, dst, dport, proto	913.3	4,096	32,436	0.087
4,096	dst, dport, proto	913.3	4,096	32,436	0.087
4,096	src, dst, proto	0.07	3	0	1.000
4,096	dst, proto	0.07	3	0	1.000
16,384	src, dst, dport, proto	776.6	16,384	14,680	0.223
16,384	src/24, dst, dport, proto	776.6	16,384	14,680	0.223
16,384	src/16, dst, dport, proto	776.6	16,384	14,680	0.223
16,384	dst, dport, proto	776.6	16,384	14,680	0.223
16,384	src, dst, proto	0.07	3	0	1.000
16,384	dst, proto	0.07	3	0	1.000

which source-prefix aggregation exists to handle, cannot be observed in this export at all. It is measured separately in the emulated network of Section VII-P, where source addresses are randomized at the packet level and the table pressure therefore falls on the source field as the reviewer’s question assumes.

VIII. ABLATION AND COMPARATIVE ANALYSIS

Two analyses that appeared in this section of the submitted manuscript have moved and changed. The feature ablation, which reported 100.00% accuracy with zero variance on every seed, was measured under the split methodology that Section VII-A corrects; Section VII-F replaces it and reaches the opposite conclusion. The leakage-robustness ablation, which removed three features chosen by inspection, is replaced by the audit in Section VII-D, which defines the suspect set by measurement. What remains here is the comparison against other methods.

A. BASELINE COMPARISON

The submitted version compared methods on a 10,000-flow class-balanced subset, which Reviewer 6 objected to on the grounds that it is not the setting the proposed detector was evaluated in. The objection is correct and the comparison has been rebuilt rather than defended. Every method is now run twice by us, on the same machine, on the CPU, under the same harness: once on the full chronological split at the natural class prevalence, and once under a subsampled protocol applied identically to every model. The comparison spans a majority-class baseline, a decision tree, logistic regression,

naive Bayes, gradient boosting in both the XGBoost [36] and LightGBM implementations, and support vector machines with linear and RBF kernels [37]. Table 19 reports the full-scale protocol, which is the one that describes deployment. One model appears only under the subsampled protocol: the RBF support vector machine, whose kernel computation is quadratic in the sample count and which cannot be fitted on 2.5 million flows on a single machine. Under the subsampled protocol, on the same CPU and harness as every other row, it reaches 94.88% macro F1 at a false positive rate of 0.00%, against 97.06% for the detector on that same protocol. We report it here rather than in Table 19 because a figure measured on 40,000 flows does not belong in a column measured on 1,077,238. For the same reason the linear support vector machine in Table 19 is fitted on a capped subsample of 300,000 flows, its liblinear solver being single-threaded and still unconverged after thirty minutes on the full partition; the cap is marked in the table and stated here because a model that cannot be trained at the deployment scale is a deployment fact rather than an inconvenience to be hidden.

Three readings follow, and only the first is favorable to us. The detector attains the joint-highest macro F1, 97.88%, statistically indistinguishable from LightGBM at 97.89%. It attains the lowest false positive rate of any method tested by a factor of four, 0.095% against 0.371% for LightGBM, which over the 1,077,238-flow test partition is 29 false alarms against 113. At an alert volume an analyst has to triage, that difference is the one that decides whether a detector is deployed.

The second reading is that per-flow latency is where the Random Forest is worst. A single-flow decision costs 7.05 ms against 0.26 ms for LightGBM. Our first explanation for that, that unbounded tree depth on 2.5 million flows produces an oversized model, turned out to be wrong when we measured it: Section VII-Q reports that the trees average 46 leaves and the whole forest serializes to 1.46 MB. The gap is a property of the per-call prediction path rather than of the model, which bounds what any change of configuration can recover.

The third is that we make no claim to a uniquely correct choice of classifier. LightGBM reaches the same macro F1 an order of magnitude faster and also admits exact TreeSHAP attribution, so an operator who values throughput over false-alarm volume should prefer it, and we say so in the text rather than leaving it for a reader to notice. What the comparison supports is narrower than the submitted version claimed: at this class ratio, on this corpus, tree ensembles separate cleanly from linear and probabilistic baselines, and the choice within the tree family is a trade between false alarms and latency rather than a matter of accuracy.

B. DEEP-LEARNING BASELINES

Reviewer 4 asked for a comparison against modern deep detectors under the same protocol. Five were implemented and run on the CPU under the subsampled protocol that Section VIII-A describes, with the same splits, the same metric definitions and the same timing harness: a 1D convolution

over the flow vector, an LSTM and a Transformer encoder over windows of sixteen consecutive flows, a reconstruction autoencoder trained on benign traffic alone, and a two-layer graph convolution over a k -nearest-neighbor graph in flow-feature space. Table 20 reports them.

Three caveats attach to individual rows rather than to the table as a whole. The sequence models classify a window rather than a flow, so their unit of decision is not the unit of enforcement. The graph model is transductive over a constructed graph, so single-flow latency is not defined for it without an incremental graph-maintenance scheme we did not implement. And the autoencoder is unsupervised, so its operating point is set differently from the rest.

On ranking quality the strongest deep model is close to the tree ensemble and none exceeds it. What separates them is the operating point. At the fixed threshold used throughout this paper the graph model classifies almost nothing as an attack despite a ROC-AUC of 0.9973, because its sigmoid outputs are compressed into a narrow band far below τ . That is a calibration failure rather than a detection failure, and reporting only the fixed-threshold column would misrepresent it, so Section VII-N reports each model at a threshold selected on held-out scores as well.

The autoencoder is the most instructive row. Trained on benign flows only, as the method requires, it assigns *lower* reconstruction error to attack flows than to benign ones: ROC-AUC 0.0073, which inverted is 0.9927. This is not an implementation defect. A randomized SYN flood is far more repetitive than benign traffic, so a model fitted to benign flows reconstructs the attack better than it reconstructs its own training distribution. The premise of reconstruction-based anomaly detection, that the attack is the anomalous class, is false on a corpus where the attack is 97% of the traffic and structurally uniform. We report it because it bears on any work applying this family of methods to this family of datasets, and because the corpus itself, rather than the method, is what fails here.

C. WHY WE DO NOT TABULATE AGAINST OTHER PAPERS

The submitted version placed our figures beside recently reported results on CIC-DDoS2019 in a table. Reviewers objected that the comparison was not a comparison, and we agree, so the table has been removed rather than re-captioned.

The reasons apply to most cross-paper tables in this literature. Kumar et al. [38] report multi-class identification of the DDoS type, which is a different and harder problem than the binary detection we perform, and an accuracy figure for one is not comparable with an accuracy figure for the other. You et al. [39] evaluate packet-level detection under adversarial perturbation, so both the unit of decision and the threat model differ. Neither uses our partition, our cleaning, or our split protocol, and Section VII-A shows that the last of those alone moves macro F1 by more than two points on identical data, which is larger than the differences such tables are usually read as establishing.

TABLE 19. Baseline comparison on the full chronological split, 2,513,556 training and 1,077,238 test flows at the natural class prevalence. All figures measured by us on the CPU of Table 5 under one harness. Latency is the median of single-flow inference calls with the estimator forced to serial execution; throughput is measured in batch mode, and the two answer different questions. [†] trained on a capped subsample of 300,000 flows because the solver does not converge on the full partition.

Method	Acc. (%)	Macro F1 (%)	FPR (%)	Lat. p50 (ms)	Flows/s	Train (s)	Exact XAI
Majority baseline	97.175	49.284	100.000	0.0058	63,672,946	0.1	×
Decision Tree	97.714	66.811	78.421	0.0304	11,775,512	10.9	×
Logistic Regression	98.350	80.873	51.134	0.1086	3,236,092	36.0	×
Naive Bayes	98.590	89.590	0.700	0.1534	2,260,331	2.9	×
XGBoost	99.515	95.802	3.299	0.4168	1,260,010	13.3	✓
Linear SVM [†]	99.641	96.825	3.289	0.1181	3,309,851	6.4	×
LightGBM	99.759	97.888	0.371	0.2551	764,706	5.9	✓
XAI-SDN (RF)	99.758	97.881	0.095	7.0480	27,647	53.3	✓

TABLE 20. Deep-learning baselines under the subsampled protocol, CPU-only, same splits and harness as Table 19. [‡] classifies a window of sixteen flows rather than a single flow. [†] transductive: single-flow latency is undefined. [¶] unsupervised; its ROC-AUC below 0.5 is discussed in the text. The detector's average precision is printed to five decimal places because it is the only entry that would otherwise round to 1.0000 and be misread as exact.

Model	Params	Macro F1 (%)	ROC-AUC	AP	Lat. p50 (ms)
1D-CNN	39,297	81.74	0.8448	0.9908	0.098
LSTM [‡]	39,489	91.02	0.9963	0.9999	0.210
Transformer [†]	73,729	81.41	0.9909	0.9997	0.360
Autoencoder [¶]	13,544	2.85	0.0073	0.9658	0.041
GNN [†]	7,809	2.80	0.9973	0.9999	0.041
XAI-SDN (RF)	200 trees	97.06	0.9987	0.99996	7.35

The only ranking claim in this paper is in Section VIII-A, where every method is run by us, on the same data, under the same protocol, on the same hardware.

IX. DISCUSSION

What follows draws out the deployment consequences of the measurements, situates the approach against deep-learning alternatives and against software-defined IoT with its additional constraints, and states plainly what the evidence does not support.

DDoS attacks on SDN controllers threaten infrastructure across enterprise, cloud, and telecommunications networks. A detector fast enough to keep pace with the control channel and transparent enough for an operator to sign off on lowers the practical barrier to deploying machine-learning defenses in these settings, where unexplained automated action is frequently the blocker rather than detection quality.

A. DEEP LEARNING FOR SDN DDOS DETECTION, AND THE TRADE IT OFFERS

Integrating any of the deep detectors of Section VIII-B into a controller is architecturally straightforward, since all of them consume the same exported flow statistics the tree ensemble consumes. The differences that matter are elsewhere. On accuracy the measured gap is small, which is the usual finding on tabular flow features. On latency it is not small and it runs

in the tree ensemble's favour. On explainability the asymmetry is structural: TreeSHAP yields exact Shapley values for a tree ensemble in time polynomial in model size [29], whereas a neural detector falls back on a model-agnostic approximation that costs more and is less faithful [8]. None of this makes deep detectors a poor choice in general. It makes them a poor fit for the position this work targets, a detector that runs inside the controller process, decides per flow, and can justify each decision cheaply enough to do so on the alert path.

B. ADAPTING XAI-SDN TO QOS-AWARE SOFTWARE-DEFINED IOT

Software-defined IoT inherits the SDN control model and adds constraints that change the design problem [20], [40]. Gateways are far weaker than the controller host used here and many are battery powered, so the binding figure is energy per decision rather than peak throughput, which favours a tree ensemble whose inference is comparisons rather than matrix arithmetic. The serialized forest is the constraint instead, and Section VII-Q shows how far it can be shrunk. Window semantics also need re-deriving: a window of N flows spans a very different interval on a sensor network than on a datacenter link, so N should be set from the intended time horizon rather than carried over. The entropy features assume concentration in an attribute is informative, which is weaker on a network where most devices legitimately talk to one endpoint. Finally, mitigation and quality-of-service policy contend for the same flow-table entries: an exact-match rule per attacking flow will evict the entries carrying QoS policy, so the aggregation analysis of Section VII-S is what makes coordinated enforcement feasible. Coupling that to a QoS-aware routing layer, as Gali and Mahamkali propose [40], is the natural extension and is not attempted here.

C. DEPLOYMENT CONSIDERATIONS

Four implications follow from the measurements, and they are not the ones the submitted version drew. Attribution, not classification, is the binding cost: classification sustains 35,715 flows/s and attributing every alert reduces that to 661, so the explanation trigger has to be budgeted rather than left implicit,

and Section VII-K gives the frontier to choose a point on. The choice of episode granularity on that frontier is a policy decision with a real cost, since a coarse episode explains the flood rather than the flow. Model size matters more than we assumed: the configuration inherited from the submitted work serializes to a forest large enough that a single-flow decision costs 7.05 ms, and Section VII-Q shows what bounding tree depth recovers. The rolling entropy engine, by contrast, is a fixed-size buffer plus hash maps and adds no measurable memory pressure. Finally, the threshold can be tuned per deployment to trade false positives against alert volume, but the score it applies to is a forest vote fraction rather than a calibrated posterior, so an operator should treat it as monotone in evidence and nothing more.

D. LIMITATIONS AND THREATS TO VALIDITY

The following bound what this paper establishes. We state them as limits on the claims rather than as directions for future work, because several of them are not resolved by anything we did.

The evaluation protocol was wrong in the submitted version of this work, and correcting it changed the headline result. The earlier manuscript described a chronological split and reported figures produced by a stratified random one. Section VII-A reports both and quantifies the difference: 97.88% macro F1 under the chronological protocol against 99.95% under the stratified one. Readers comparing against other published results on this corpus should assume that a comparable gap may separate reported figures from what a chronological protocol would yield, unless the paper in question states its protocol.

Dataset-specific artifacts cannot be excluded. The leakage audit in Section VII-D shows that removing every objectively flagged suspect feature leaves performance unchanged, that no feature has disjoint class ranges, and that no test flow has an identical training twin after deduplication. These results are inconsistent with several specific artifact mechanisms. They do not establish that the corpus is free of artifacts, and no feasible experiment on a single synthetic corpus could. We say this explicitly because the earlier version of this work claimed that a three-feature ablation confirmed the absence of leakage, which it did not.

Attack coverage is partial. The primary evaluation is a SYN flood. Section VII-J extends to other reflection and amplification vectors in the same corpus and to a different capture day, and Section VII-J to two further corpora. Performance degrades under transfer, and we report the degradation rather than the best case. Application-layer attacks, low-rate attacks, and slow-read attacks are not evaluated at all, and nothing here should be read as evidence about them.

The testbed is small. Section VII-R runs the detector inside a controller process exchanging genuine OpenFlow 1.3 messages with switch agents over TCP, and Section VII-P runs it against an emulated network of three hosts and one Open vSwitch datapath under Mininet. Both are real control planes and neither is a network of operational scale. A single

switch cannot exhibit multi-datapath rule distribution, controller failover, or the east-west traffic patterns of a datacenter fabric, and the QoS figures we report describe contention at one controller and one 100 Mbit link rather than forwarding-path behavior under production load.

Transferability is bounded by labeled minority traffic, not by labeled volume. Section VII-J1 shows that a hundred labeled flows from one target network are enough to reach 0.9964 macro F1, while fifty thousand from another are not enough to reach 0.90. The difference is the number of benign examples the target's own capture contains. An operator cannot infer the labelling cost of deployment from the flow count alone, and we cannot predict it from anything we measured.

The adaptive adversary is out of scope. The threat model assumes an attacker who floods but does not optimize against the detector. An adversary who knows the feature set can rotate spoofed sources to hold per-window entropy near its benign value, and the distributional evidence degrades accordingly. We have not measured what that evasion costs the attacker in attack effectiveness, which is the quantity that would determine whether the defense remains useful under adaptation. This is the most consequential gap in the paper.

Budgeted alerts do not carry an exact attribution of their own score. Under any policy that admits fewer alerts than it receives, the alerts the budget declines carry the attribution of their episode representative, $\phi_{p(i)}$ in Eq. (4). That vector satisfies the efficiency property of Eq. (3) for the representative flow and not, in general, for the flow it is attached to, so it identifies why the episode was blocked rather than why this flow was scored as it was. At the per-service granularity that recovers the classification rate, that applies to all but two of 59,998 alerts. An operator who needs a per-flow answer must tighten the granularity or re-explain the flow on demand, and the frontier in Section VII-K is the cost of doing so. We flag this because the unbudgeted reading, in which every alert carries its own exact attribution, is the one a reader would otherwise carry over from the abstract.

Detector confidence is uncalibrated. The score is a forest vote fraction. It is monotone in evidence and adequate for thresholding, but it is not a calibrated posterior, and the alert payload presents it to an operator as though it were interpretable on its own scale. Bayesian treatment of this quantity [31] would give the operator something better to act on at borderline scores, and is not attempted here.

X. CONCLUSION

XAI-SDN is a controller-embedded DDoS detector that augments flow statistics with windowed Shannon entropy maintained at expected amortized $\mathcal{O}(1)$ cost, classifies with a Random Forest, and attributes alerts with exact TreeSHAP under an explanation budget, every admitted alert receiving an attribution of its own score and every other resolving to its episode representative. Under a chronological split of the CIC-DDoS2019 SYN partition it reaches 99.76% accuracy and 97.88% macro F1 over 1,077,238 held-out flows, at a

false positive rate of 0.095%, the lowest of any method we evaluated.

The detection figure is the least interesting result here. Three others matter more to anyone intending to deploy such a system. Explanation is affordable only if its trigger is budgeted, since attributing every positive prediction collapses sustained throughput when almost all traffic is hostile, and the throughput-versus-coverage frontier we report lets an operator choose a point on it rather than discover the cost in production. Control-plane cost is not inference cost: measured inside a live OpenFlow controller, the binding constraints are the rate at which table-miss events can be ingested and the rate at which mitigation rules exhaust a bounded flow table, neither of which appears in an offline throughput number. And the evaluation protocol moves the headline metrics on this benchmark further than any architectural choice we examined, which is a caution that applies well beyond this paper.

We are equally explicit about what the evidence does not support. The entropy augmentation does not improve indistribution detection on this corpus, and buys robustness under shift only where its full feature set is computable. The detector does not transfer to another corpus without retraining, and on one of the two we tried, an operator is better off discarding it and labelling a hundred of their own flows. No experiment available to us can exclude dataset-specific artifacts. Section IX-D states these as limits on the claims rather than as future work. The work we consider most pressing follows from them: quantifying what evasion costs an adaptive adversary who shapes per-window distributions, calibrating the detector score so that borderline alerts carry usable confidence, and evaluating on captures whose benign traffic is not confined to a single interval of the recording.

DECLARATION OF COMPETING INTEREST

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support provided by the Higher Colleges of Technology (HCT), United Arab Emirates, toward the publication of this research in the IEEE Access journal. This support contributed significantly to the successful dissemination of the research findings.

DATA AVAILABILITY

The datasets, implementation code, trained-model artifacts, per-seed manifests, and experimental configuration files supporting this study are publicly available at: <https://github.com/adeliusa486/XAI-SDN>

REFERENCES

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [2] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmoly, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.
- [3] J. Mirkovic and P. Reiher, "A taxonomy of DDoS attack and DDoS defense mechanisms," *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 2, pp. 39–53, 2004.
- [4] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic Distributed Denial of Service (DDoS) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCSST)*. IEEE, 2019, pp. 1–8.
- [5] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network anomaly detection: Methods, systems and tools," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, pp. 303–336, 2014.
- [6] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82–115, 2020.
- [7] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 93:1–93:42, 2018.
- [8] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck, "Evaluating explanation methods for deep learning in security," in *Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2020, pp. 158–174.
- [9] L. S. Al and Ş. Sağiroğlu, "Explainable artificial intelligence models in intrusion detection systems," *Engineering Applications of Artificial Intelligence*, vol. 144, p. 110145, 2025.
- [10] S. M. Mousavi and M. St-Hilaire, "Early detection of DDoS attacks against SDN controllers," in *Proceedings of the 2015 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, 2015, pp. 77–81.
- [11] A. K. Jain, H. Shukla, and D. Goel, "A comprehensive survey on DDoS detection, mitigation, and defense strategies in software-defined networks," *Cluster Computing*, vol. 27, no. 9, pp. 13 129–13 164, 2024.
- [12] G. Nychis, V. Sekar, D. G. Andersen, H. Kim, and H. Zhang, "An empirical evaluation of entropy-based traffic anomaly detection," in *Proceedings of the 8th ACM SIGCOMM Conference on Internet Measurement (IMC 2008)*. ACM, 2008, pp. 151–156.
- [13] A. Lakhina, M. Crovella, and C. Diot, "Mining anomalies using traffic feature distributions," in *Proceedings of the ACM SIGCOMM 2005 Conference*. ACM, 2005, pp. 217–228.
- [14] A. Lall, V. Sekar, M. Ogihara, J. Xu, and H. Zhang, "Data streaming algorithms for estimating entropy of network traffic," in *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. ACM, 2006, pp. 145–156.
- [15] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proceedings of the 35th Annual IEEE Conference on Local Computer Networks (LCN)*. IEEE, 2010, pp. 408–415.
- [16] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep learning approach for network intrusion detection in software defined networking," in *Proceedings of the 2016 International Conference on Wireless Networks and Mobile Communications (WINCOM)*. IEEE, 2016, pp. 258–263.
- [17] Q. Niyaz, W. Sun, and A. Y. Javaid, "A deep learning based DDoS detection system in software-defined networking (SDN)," *EAI Endorsed Transactions on Security and Safety*, vol. 4, no. 12, 2017.
- [18] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2022, pp. 1–9.
- [19] L. D. Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilake, M. Sarhan, and M. Portmann, "FlowTransformer: A transformer framework for flow-based network intrusion detection systems," *Expert Systems with Applications*, vol. 241, p. 122564, 2024.
- [20] J. Bhayo, S. A. Shah, S. Hameed, A. Ahmed, J. Nasir, and D. Draheim, "Towards a machine learning-based framework for DDoS attack detection in software-defined IoT (SD-IoT) networks," *Engineering Applications of Artificial Intelligence*, vol. 123, p. 106432, 2023.
- [21] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. SciTePress, 2018, pp. 108–116.

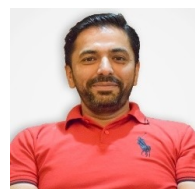
- [22] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A novel SDN intrusion dataset," *IEEE Access*, vol. 8, pp. 165 263–165 284, 2020.
- [23] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.
- [24] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*. IEEE, 2010, pp. 305–316.
- [25] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressneger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proceedings of the 31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988.
- [26] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv:1702.08608 [stat.ML], 2017, preprint.
- [27] M. T. Ribeiro, S. Singh, and C. Guestrin, "“Why should I trust you?”: Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1135–1144.
- [28] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Curran Associates, Inc., 2017, pp. 4765–4774.
- [29] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.
- [30] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron," *IEEE Access*, vol. 12, pp. 30 164–30 175, 2024.
- [31] M. Swaminathan, O. W. Bhatti, Y. Guo, E. Huang, and O. Akinwande, "Bayesian learning for uncertainty quantification, optimization, and inverse design," *IEEE Transactions on Microwave Theory and Techniques*, vol. 70, no. 11, pp. 4620–4634, 2022.
- [32] F. Chen, C. Ding, X. Hu, X. He, X. Yin, J. Yang, and Z. Zhao, "Tensor Poincaré plot index: A novel nonlinear dynamic method for extracting abnormal state information of pumped storage units," *Reliability Engineering & System Safety*, vol. 254, no. Part B, p. 110607, 2025.
- [33] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [34] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [36] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [37] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [38] A. I. Kumar, G. Ishigaki, and A. K. Belman, "Enhanced multi-class DDoS attack identification using a meta-learning ensemble," arXiv:2607.16521 [cs.CR], 2026, preprint.
- [39] J. You, Z. Zhou, S. Yang, Y. Sun, L. Zhang, and C. Xu, "PLAA: Packet-level adversarial attacks in network traffic detection," arXiv:2606.28439 [cs.CR], 2026, preprint.
- [40] M. Gali and A. Mahamkali, "Deep learning based attack detection and QoS-aware secure routing protocol for SDN-IoT network," *Concurrency and Computation: Practice and Experience*, vol. 37, no. 4–5, p. e70045, 2025.



ADEEL AHMAD (Student Member, IEEE) is currently pursuing the Bachelor's degree in information technology at the Islamic University of Madinah, Saudi Arabia, where he is a recipient of a Saudi Government-funded scholarship for academic excellence. His research interests include cybersecurity, software-defined networking, machine learning, and cloud computing. He is actively involved in applied machine learning research, including explainable AI for DDoS detection and privacy-preserving federated learning. He has authored research accepted at the IEEE Vehicular Technology Conference (VTC2026-Fall).



ARSHAD ALI (Senior Member IEEE) joined Islamic University of Madinah in December 2014, he is promoted as full Professor of Data Communication and Networking, in Faculty of Computer and Information Systems, Islamic University of Madinah. He joined Aston University, Birmingham, UK in 2005 and obtained his MSc Telecommunication Technology in 2007. In 2007, he joined Geotechnical Group, Department of Engineering, and University of Cambridge as Research Ass. (2007- 2009). In 2009, he was awarded a PhD (2009- 2012) scholarship from Lancaster University, UK and he was awarded PhD in 2012. With over 18 years of teaching experience, he has taught more than 13 computing courses across information technology, cybersecurity, and networking, mentoring over 15 graduate students. His research interests include cybersecurity, wireless sensor, networking, and Artificial Intelligence He has published over 90 peer-reviewed articles in reputed journals. He is heading the Department QAU for accreditation and is an ABET Information Technology Program Evaluator. His teaching and mentoring have earned him several awards, including the "Faculty Excellence Award 2018 at IUM. He worked on the UK-NEES project and designed communication systems for live experimentation between UK Universities (Cambridge, Oxford and Bristol).



RAFI US SHAN, PhD (Senior Fellow IEEE) is serving as faculty and served as division chair at Higher Colleges of Technology, Abu Dhabi UAE. Mr. Shan has served as project director Digital Transformation Cell at Ministry of IT, Pakistan and Chief Cyber Security at Khyber Pakhtunkhwa Cyber Emergency Response Center (KP-CERC). Mr. Shan has contributed to national-level ICT and cybersecurity policy development, led the design and deployment of major ICT initiatives, and provided advisory services to provincial and national organizations. He has served on the President of Pakistan's Information Security Committee, the National Center for Cyber Security (NCCS) Evaluation Board, and several other national and international ICT and cybersecurity forums. He has served at COMSATS University Islamabad. After his MS in personal mobile and radio communication he completed his PhD at The University of Lancaster, UK.

**GAHANGIR HOSSAIN** (Senior Member, IEEE)

received the bachelor's degree in computer science and engineering and the dual master's degree in computer engineering (focused on cognitive neuroscience) and in computer science (focused on cybersecurity) and the Ph.D. degree in computer engineering from the University of Memphis, where he focused on computer science, cognitive science, and information systems. With over 20 years of teaching experience, he has taught more than 18

computing courses across computer science, cybersecurity, and biomedical engineering, mentoring over 20 graduate students, including three Ph.D. candidates. His research interests include cybersecurity, data science, cognitive neuroscience, AI, and recommender systems. He has published over 150 peer-reviewed articles and directs the SCORE Laboratory, UNT. He has secured over \$3 million in research funding, currently leading two major projects on cybersecurity research, education, and training. He also serves as an Associate Editor for several journals, has held leadership roles like Director of the Cyber Security Laboratory and is an ABET Computing Program Evaluator. His teaching and mentoring have earned him several awards, including the "Faculty Excellence Award 2023" at UNT and the "Best Paper" Award at IEEE CCWC 2024.